



**UNIVERSITY
OF LONDON**

CM3070 Final Project Report

Ryan Aw 240662206

ARIA – Activity Recognition & Inference Architecture: A Privacy-Preserving Multimodal Activity Recognition and Camera-Local Tracking System for a Shared Service-Counter Environment

Template: CM3065 13.2 – Multi-human Detection and Activities Tracking in a Shared Space

Code Repository: [<https://github.com/emulsifiedCheese/aria>]

Table of Contents

1. Introduction.....	5
1.1. Project Concept and Motivation.....	5
1.2. Scope and Research Questions.....	5
1.3. Current Project Status.....	6
2. Literature Review.....	7
2.1. Multimodal Sensing and Edge Inference.....	7
2.2. Multi-User Activity Recognition Architectures.....	7
2.3. Audio-Vision Fusion Strategy.....	8
2.4. Dataset Construction Methodology.....	8
2.5. Core Tracking Algorithm.....	9
2.6. Synthesis and Critical Comparison.....	9
3. Design.....	11
3.1. Requirements and Active Scope.....	11
3.2. System Architecture.....	11
3.3. Privacy-First Collection Design.....	12
3.4. Temporal Alignment and Feature Design.....	13
3.5. Activity Taxonomy and Evaluation Design.....	15
3.6. Operational Resilience and Data Integrity.....	16
4. Implementation.....	17
4.1. ESP Telemetry and Ingestion.....	17
4.2. Camera, Masking, and Vision Pipeline.....	18
4.3. Tracking, Pose, and Feature Records.....	19
4.4. Multimodal Window Construction.....	21
4.5. Session Lifecycle and Annotation.....	22
4.6. Audited Dataset and Frozen Model.....	23
4.7. Local Inference, Dashboard, and Firebase Projection.....	25
4.8. Frozen Release and Reproducibility.....	28
4.9. Implementation Testing and Technical Challenges.....	28
5. Evaluation.....	29
5.1. Evaluation Strategy and Justification.....	29
5.2. Technical Evaluation and Testing Types.....	29
5.3. Sensor, Camera, and Runtime Reliability.....	30
5.4. Audited Dataset and Split Integrity.....	31
5.5. Development Ablation and Model Selection.....	34
5.6. One-Time External Evaluation and Failure Analysis.....	35
5.7. Privacy, Degraded Modes, and Delivery.....	37
5.8. Critical Evaluation Against the Research Questions.....	38
6. Conclusion.....	39
6.1. Project Summary and Achievement of Objectives.....	39
6.2. Limitations and Future Work.....	39
6.3. Self-Learning and Reflection.....	39
References.....	40

Index of Tables

Table 1: Active scope and intended outputs.....	6
Table 2: Literature contributions, limitations, and ARIA response.....	9
Table 3: Active hardware and data streams.....	12
Table 4: The 23 input fields used by the Camera-only classifier.....	14

Table 5: Activity taxonomy and expected evidence.....	15
Table 6: Principal implementation modules.....	20
Table 7: Final verification evidence.....	29
Table 8: Selected live reliability and runtime results.....	30
Table 9: Controlled tracking observations and limits.....	31
Table 10: Final audited modelling dataset.....	31
Table 11: Dev-only modality ablation across 8 frozen LOSO folds. <i>Differences are calculated before rounding</i>	34
Table 12: Frozen Camera-only external-test results.....	35
Table 13: Per-class external performance.....	35
Table 14: Held-out performance against constant-class reference.....	37
Table 15: Research-question outcomes.....	38

Table of Figures

Figure 1: Active Zone A system architecture.....	11
Figure 2: Fail-closed collection lifecycle.....	13
Figure 3: Deterministic alignment into a two-second multimodal window.....	14
Figure 4: Active-node and schema validation in /src/aria/ingestion/packet_parser.py.....	17
Figure 5: ESP-A1 with PIR, ultrasonic, and microphone sensors.....	17
Figure 6: Creation of masked frame in /src/aria/cameras/privacy_mask.py.....	18
Figure 7: Batched pose estimation and coordinate restoration in /src/aria/vision/pose_estimator.py.....	19
Figure 8: Experimental anonymous stitching branches (not deployed) in /src/aria/vision/track_stitcher.py. .	20
Figure 9: Deterministic window grouping in /src/aria/fusion/window_builder.py.....	21
Figure 10: Worked feature-to-prediction example using controlled numerical pose inputs.....	22
Figure 11: Fail-closed handling of a missing camera-local track in /src/aria/collection/live_annotation.py...	23
Figure 12: Fold-local preprocessing and Random Forest fitting in /src/aria/modeling/baselines.py.....	24
Figure 13: Dashboard extracts during controlled tests: ready state with activity displayed (top), and blocked state with activity cleared and privacy and stale-input warnings visible (bottom).....	25
Figure 14: Fail-closed prediction output and schema validation in /src/aria/inference/service.py.....	26
Figure 15: Privacy-bounded Firebase payload projection in /src/aria/cloud/firebase_sync.py.....	27
Figure 16: Observed dashboard states during a controlled mask-failure and recovery check.....	27
Figure 17: Audited activity distribution after quality exclusions.....	32
Figure 18: Session and participant separation in development and the held-out test.....	33
Figure 19: Class proportions in development and the held-out test; labels include row counts.....	33
Figure 20: Frozen dev modality ablation; Camera A-only remains the selected model.....	34
Figure 21: Retained fold-level macro F1 for Camera-only and Camera + ESP-A1.....	35
Figure 22: External-test confusion matrix for the frozen Camera-only model.....	36

List of Abbreviations

Abbreviation	Meaning
1D-CNN	One-Dimensional Convolutional Neural Network
ARIA	Activity Recognition & Inference Architecture
BSN	Body Sensor Network
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CV	Computer Vision
ESP8266	Espressif ESP8266 Wi-Fi microcontroller
F1	Harmonic mean of precision and recall

FPS	Frames Per Second
H.264	H.264 video-compression standard
HAR	Human Activity Recognition
HOTA	Higher Order Tracking Accuracy
ID	Identifier
IDF1	Identification F1 Score
JSON	JavaScript Object Notation
JSONL	JSON Lines
LOSO	Leave-One-Session-Out
MOTA	Multiple Object Tracking Accuracy
MOT17	MOTChallenge 2017 multi-object tracking benchmark
PIR	Passive Infrared
p50	50 th percentile (median)
p95	95 th percentile
RFID	Radio-Frequency Identification
RSSI	Received Signal Strength Indicator
RTMP	Real-Time Messaging Protocol
RTSP	Real-Time Streaming Protocol
SGT	Singapore Time
SHA-256	Secure Hash Algorithm with 256-bit digest
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
UTF-8	Unicode Transformation Format, 8-bit
YOLO	You Only Look Once

Section	Word Count
Introduction (1000)	771
Literature Review (2500)	1649
Design (2000)	1425
Implementation (2500)	2350
Evaluation (2500)	1870
Conclusion (1000)	390
Total (Max 10500)	8455

1. Introduction 771/1000

1.1. Project Concept and Motivation

ARIA (Activity Recognition & Inference Architecture) is a privacy-preserving multimodal system designed to recognise the activities of consenting Counter Agents in a shared service-counter environment. It combines a staff-facing camera with two ESP8266 sensor nodes, aligns the resulting measurements into two-second windows, and classifies activity locally using person-linked visual features. The project follows CM3065 Template 13.2, Multi-human Detection and Activities Tracking in a Shared Space, adapted to concurrent consenting staff while excluding customers and other non-participants.

The project addresses a distinction that ordinary occupancy data cannot capture. A counter may be occupied while staff are serving or processing a transaction, waiting for work, or reaching for documents and equipment. Computer-activity logs reveal only part of this behaviour, while continuous manual observation is impractical and intrusive. ARIA therefore examines whether low-cost passive sensing can distinguish these activities without wearables, retained speech, or cloud-hosted video.

ARIA's main technical challenge is not simply combining sensors, but deciding what each sensor can legitimately reveal. Vision can associate pose and movement with a camera-local track, yet this association may fail during overlap, occlusion, or visually similar postures. PIR, ultrasonic, and numerical microphone measurements describe the shared zone rather than a particular person. The implementation must therefore align asynchronous evidence, preserve missing data, prevent participant or session leakage, and test whether fusion improves classification rather than assuming that it will.

ARIA is a research and decision-support system, not an individual performance-surveillance tool. Its stakeholders are consenting Counter Agents, the researcher and the workplace organisation. Records remain local and pseudonymous, and outputs are intended for aggregate evaluation rather than disciplinary decisions or persistent employee profiles.

A live service counter creates conditions that are absent from many laboratory datasets: close seating, brief and overlapping activities, furniture occlusion, wireless packet loss, and limited laptop processing capacity. Customers and other non-participants may also enter nearby areas despite being outside the study. ARIA must therefore be evaluated not only by classification accuracy, but also by acquisition reliability, tracking stability, honest treatment of missing data, and privacy controls that fail safely.

The project makes two linked contributions. Empirically, it tests whether masked vision benefits from the addition of numerical audio, PIR, and ultrasonic evidence. Technically, it demonstrates how these streams can be collected, aligned, and evaluated locally within a privacy-preserving design. Success therefore depends on more than an overall score: it requires per-class results, credible person association, acceptable latency, transparent failure analysis, and evidence that the privacy boundary remains effective during both normal operation and faults.

1.2. Scope and Research Questions

The final study is deliberately limited to Zone A, ESP-A1, ESP-A2 and the DJI Osmo Action 5 Pro stream named camera_a. Only consenting Counter Agents are research participants. Station Manager attribution, Zone B, Zone T, ESP-B, ESP-T and a second camera are outside the implementation, collection and

acceptance criteria. Person association is session-local and pseudonymous; ARIA does not attempt face recognition, biometric identification or persistent recognition across sessions.

Table 1: Active scope and intended outputs

Area	Active Project Scope	Explicit Exclusion
People	Consenting Counter Agents identified only by pseudonymous codes, session-local tracks	Customers, Station Managers, non-participants, biometric identity
Place	Zone A staff-side counter region	Zone B, Zone T, customer-facing regions
Sensors	ESP-A1, ESP-A2, camera_a	ESP-B, ESP-T, built-in cameras, second-camera sources
Retained Data	Validated telemetry, pose/movement features, annotations, manifests, predictions	Raw audio, intelligible speech, cloud-hosted video
Primary Result	Per-class activity recognition and comparison of camera, sensor-node and fused configurations	Role attribution between Counter Agent and Station Manager

The evaluation is guided by four research questions: (1) does multimodal fusion improve per-class and macro F1 over vision-only and sensor-based alternatives; (2) can sensing, vision, and fusion operate with acceptable latency and availability on the local laptop; (3) how stable are camera-local identities during ordinary work, overlap, occlusion, exit, and re-entry; and (4) do the privacy and lifecycle controls prevent collection when consent, masking, camera, sensor, or storage conditions are invalid?

1.3. Current Project Status

The completed study comprises nine retained formal sessions and 8 hours, 00 minutes, and 20 seconds of recorded activity. Reconstruction produced 28,139 participant-window annotations, of which 26,114 participant-level rows across 1,847 activity episodes remained after uncertain, track-absent, and conflicting assignments were excluded. The final modelling data contain 24,046 rows in eight leave-one-session-out development folds and 2,068 rows in a participant- and session-isolated external test.

The results do not support the original expectation that multimodal fusion would outperform vision alone. Camera A-only achieved the highest development macro F1 at 0.443, while every tested sensor-only and early-fusion alternative performed worse. The selected Camera-only model was then evaluated once on the isolated external test, achieving 0.492 accuracy, 0.328 balanced accuracy, and 0.348 macro F1. It failed to recognise any of the 19 external Reaching/Handling rows, so reliable performance cannot be claimed for that class.

At system level, the local inference service, privacy-aware dashboard, restricted Firebase output, and reproducible release all passed their final functional and privacy checks. The 5,400-second stability run produced a more qualified result: system health remained degraded for most of the run, with estimated ESP-A1 and ESP-A2 delivery of 91.70% and 91.50%. The implementation passed the tested application-level privacy checks and the clean-extraction software demonstration. Classification, tracking, network reliability, sample size and generalisability remain important limitations.

2. Literature Review 1649/2500

This review examines multimodal edge inference, workplace sensing, multi-user architecture, audio-video fusion, dataset construction and tracking. Additional methodological sources explain the final classifier, pose representation and validation design. The discussion considers both the contribution of each approach and the limits of transferring results to a live shared counter.

2.1. Multimodal Sensing and Edge Inference

Nakabayashi and Saito [6] fused camera images with inertial measurements in a lightweight attention-based network and reported approximately 97% precision across five activities on the UESTC-MMEA-CL dataset. Their work demonstrates that heterogeneous sensor fusion can outperform single-sensor baselines while remaining suitable for resource-constrained edge devices. This supports ARIA's decision to align vision, numerical audio, PIR and ultrasonic evidence locally rather than depend on cloud inference.

However, the reported performance cannot be transferred directly to ARIA. The sensing configuration is different, the evaluation is largely single-user, and the study does not address identity disambiguation within a shared camera view. It therefore motivates ARIA's multimodal hypothesis without proving it; that claim must be tested on the same grouped data splits for vision-only, sensor-only, and fused models.

Golipoor and Sigg [3] evaluated multimodal convolutional networks using body-worn RFID measurements collected in an indoor office, motivated by applications in industrial safety and automation. Their work is valuable because it shows that non-visual sensing can contribute to activity recognition in a professional environment and motivates temporal neural models as an alternative. ARIA did not evaluate a 1D-CNN, so no claim is made that Random Forest outperforms that architecture. Its central limitation is the body-worn RFID dependency. A wearable uniquely links measurements to a participant, whereas ARIA's ambient sensors are passive and shared. ARIA gains deployment convenience but inherits the harder problem of deciding when zone-level evidence can legitimately support person-linked classification.

2.2. Multi-User Activity Recognition Architectures

Li et al. [4] provide architectural precedent for separating resource-intensive training from lightweight edge inference in a multi-user body-sensor network. For ARIA, the important principle is that latency-sensitive and privacy-sensitive processing can remain local while carefully limited, abstract outputs support later synchronisation. ARIA applies this separation through laptop-side capture, masking, feature extraction, and inference, with Firebase restricted to non-identifying summaries.

The similarity is architectural rather than sensor-level. Li et al. [4] use wearables, so each input is already user-linked; ARIA uses camera-local tracks and shared ambient sensors. It must solve attribution without worn hardware and continue locally without cloud connectivity. Video and raw audio remain outside the cloud boundary.

Arrotta et al. [1] formalise multi-subject human activity recognition and identify simultaneous, potentially overlapping activities as an unresolved extension of mature single-subject HAR. They highlight shortages in multi-subject datasets, underdeveloped per-subject evaluation and the difficulty of disambiguating activity in shared spaces. This directly frames ARIA's central challenge: a correct class assigned to the wrong camera-local track is still a system failure.

The survey supports per-class F1, confusion matrices and participant- or session-separated validation instead of accuracy on randomly mixed windows. ARIA treats classification and identity continuity as separate outcomes, while explicitly limiting claims because of its narrow ethical scope, class imbalance and ambient-signal ambiguity.

2.3. Audio-Vision Fusion Strategy

Cristina et al. [2] review audio-, video- and multimodal activity-recognition systems in healthcare, examining performance, complexity, robustness, scalability and privacy. They identify limited real-world validation and small datasets as barriers to generalisation. Their review provides a basis for considering complementary sensing while emphasising that its usefulness must be evaluated in the intended setting.

The healthcare evidence is useful but not directly transferable to ARIA’s shared-counter setting. For ARIA, it motivates testing whether numerical acoustic activity provides useful context when staff activities have similar visual appearances. However, the microphone features cannot identify which Counter Agent produced a sound, and no intelligible speech is retained. Audio measurements therefore provide shared zone-level context within each two-second window, rather than direct evidence of an individual’s activity. The review motivates testing fusion but does not establish that ambient audio will improve person-level recognition in this setting.

In response to the privacy concerns discussed by Cristina et al. [2], ARIA keeps visual processing local, applies the approved privacy mask before visual analysis, removes camera audio and retains only numerical microphone features. These are project-specific design decisions for a semi-public workplace.

2.4. Dataset Construction Methodology

Moreaux et al. [5] introduced Kitchen20, a dataset of 800 five-second environmental-audio clips across 20 classes. Clips were obtained from FreeSound and additional kitchen recordings, with documented class definitions, predefined cross-validation folds and human assessment. The study illustrates the value of consistent samples, source provenance and reproducible benchmarking.

ARIA applies these general principles through defined activity classes, timestamped annotations and documented collection conditions. Its annotation method is a project-specific adaptation: rather than classifying isolated sound clips, it assigns activities to camera-local tracks during continuous, concurrent workplace activity. This introduces additional challenges from occlusion, ambiguous transitions, missing packets and class imbalance.

Applying these principles in Zone A requires additional safeguards. Uncertain and excluded intervals remain traceable, participant and session groupings are preserved during data splitting, and shared sensor features are never presented as independently measured person-level observations. The resulting dataset is therefore appropriate for evaluating ARIA’s research questions within its consent, privacy, and distribution constraints, but it is not claimed as a general-purpose public benchmark.

Dehghani et al. [9] show that human activity recognition results depend on whether validation separates subjects, because observations from the same person are correlated. This matters even with ARIA’s non-overlapping windows: repeated observations within an episode are not independent examples. ARIA therefore separates sessions during development and reserves different participants for the final test. Its repeated development participants still limit what cross-validation can establish about unfamiliar people.

2.5. Core Tracking Algorithm

Zhang et al. [7] propose ByteTrack, which associates both high- and low-confidence detections with existing tracklets instead of discarding every low-score box. On MOT17 they report 80.3 MOTA, 77.3 IDF1 and 63.1 HOTA, showing that low-score association can reduce fragmentation when people are partly occluded. ARIA uses ByteTrack through the vision subsystem because close seating, counter fixtures and overlap can temporarily lower detection confidence.

ByteTrack is an enabler rather than an activity classifier. It supplies camera-local identities to which pose and later activity evidence can be attached. Its benchmark results also cannot be treated as ARIA results: MOT17 uses different cameras, scenes and computing hardware. Controlled Zone A trials have already recorded swaps, duplicate IDs and fragmented tracks. Identity stability must therefore be measured in the deployment geometry using transitions, fragmentation, recovery and manually checked association metrics.

Tracking errors propagate: fragmentation divides an interval across identifiers, while an identity switch can attach valid evidence to the wrong participant. These differ from classification errors and require separate diagnosis. Zhang et al.'s [7] metrics provide reference measures, but Zone A analysis must also stratify close seating, occlusion, similar posture, exit and re-entry, and lost keypoints to distinguish feature-classification errors from earlier association failures.

Ultralytics describes pose outputs as body-keypoint coordinates with confidence information [10]. ARIA uses these numerical outputs as an intermediate representation rather than storing images or assuming that a detected pose identifies an activity. Confidence filtering and camera-local association are necessary because occlusion can remove joints and adjacent people can enter the same crop. These deployment-specific errors remain separate from the pose model's benchmark performance.

2.6. Synthesis and Critical Comparison

Table 2: Literature contributions, limitations, and ARIA response

Source	Contribution	Limitation for this project	ARIA response
Nakabayashi and Saito [6]	Multimodal edge HAR	Single-user setting and different sensors	Test identical modality baselines locally
Golipoor and Sigg [3]	Non-visual workplace sensing and CNNs	Wearable RFID identifies the user	Evaluate passive ambient features and attribution limits
Li et al. [4]	Multi-user edge-cloud separation	Wearable BSN and different privacy boundary	Keep inference local; sync validated prediction summaries only
Arrotta et al. [1]	Multi-subject HAR framework	Survey does not solve deployment tracking	Use grouped F1 and separate identity evaluation
Cristina et al. [2]	Review of healthcare HAR and deployment limitations	Shared audio does not identify a person	Separate zone evidence from person attribution

Source	Contribution	Limitation for this project	ARIA response
Moreaux et al. [5]	Domain-specific dataset methodology	Isolated audio clips; no multi-person identity tracking	Retain provenance, uncertainty and multimodal timing
Zhang et al. [7]	Low-score detection association	Benchmark geometry differs from Zone A	Measure fragmentation and recovery on-site

Across these sources, the most persistent gap is the transition from single-user or wearable-linked recognition to passive multi-person attribution. Strong single-user fusion results do not resolve which person generated a shared ambient signal. Wearable BSN and RFID systems simplify identity but impose cooperation requirements that ARIA deliberately avoids. ByteTrack supplies a visual association mechanism, yet the deployment trials show that close overlap and occlusion can still fragment identity. The literature therefore supports ARIA's problem formulation while also explaining why tracking stability must be evaluated independently of classification.

A second theme is the privacy-edge relationship. Li et al. [4] establish a useful division between edge inference and later training, while Cristina et al. [2] identify privacy and scalability as deployment concerns. ARIA makes the privacy boundary operational: the mask precedes all vision consumers, camera audio is removed, raw audio is never retained and cloud synchronisation is restricted to validated prediction summaries. Edge execution is necessary but not sufficient; fail-closed controls are needed to prevent invalid inputs or privacy incidents from entering the dataset.

Wearable approaches [3, 4] impose hardware but directly link measurements to users. ARIA avoids that burden, while accepting that microphone, PIR and ultrasonic evidence describes a shared zone. This creates two distinct problems: synchronisation establishes which measurements overlap in time, whereas attribution establishes whose activity they describe. A common window provides a reproducible temporal grouping, but does not establish either exact physical synchronisation or who generated each measurement. Copying the same ambient features to simultaneous person-level rows does not create independent evidence for each person. Fusion therefore needs complementary information at the appropriate spatial and temporal level, not simply additional input fields.

Finally, Moreaux et al. [5] and Arrotta et al. [1] show why domain-specific data and multi-subject evaluation matter. ARIA contributes an implemented privacy-first collection and inference system together with a documented Zone A dataset for three Counter Agent activity classes. Its originality lies in combining ambient sensing, masked local vision, camera-local multi-person tracking, deterministic reconstruction and auditable incident handling in one workplace deployment. The completed ablation also qualifies the literature's multimodal expectation: adding ambient node features did not improve macro F1 over Camera-only. This negative result highlights the difficulty of combining person-linked visual features with shared zone-level measurements, although the experiments do not isolate the cause of the performance reduction.

Breiman's Random Forest combines randomised decision trees [8], providing a basis for modelling nonlinear relationships among ARIA's numerical window features. For this small, correlated dataset, a feature-based classifier also permits the same training procedure to be applied to each modality subset. This is a practical design justification, not evidence of superiority over temporal neural networks. Window averaging may discard brief action order, and adding weak or shared-context features can still reduce generalisation. The final ablation therefore tests a specific classifier and representation rather than multimodal recognition in general.

3. Design 1425/2000

3.1. Requirements and Active Scope

Five requirements shape the final design. ARIA must accept only the approved Zone A camera and sensor nodes, apply the verified privacy mask before vision processing, retain no raw audio, preserve missing data during multimodal alignment, and keep participant identity pseudonymous and local. If the camera, mask, sensors, consent state, or storage no longer matches the approved configuration, collection must pause rather than continue with uncertain or unsafe inputs.

The final activity taxonomy contains Serving/Processing, Idle/Waiting, and Reaching/Handling. Serving and Typing/Processing were merged after the pilot because ordinary counter work did not provide a consistent observable boundary between them and their postures were often similar. Original annotations were preserved, while the combined label was applied only in the derived modelling data so that the change remains traceable.

One camera and two low-cost ambient nodes limit spatial coverage but keep each measurement traceable. Avoiding wearables reduces disruption while making person attribution harder, and local processing trades laptop capacity for a stronger media boundary. These constraints align the design with the workplace and participant protections.

3.2. System Architecture

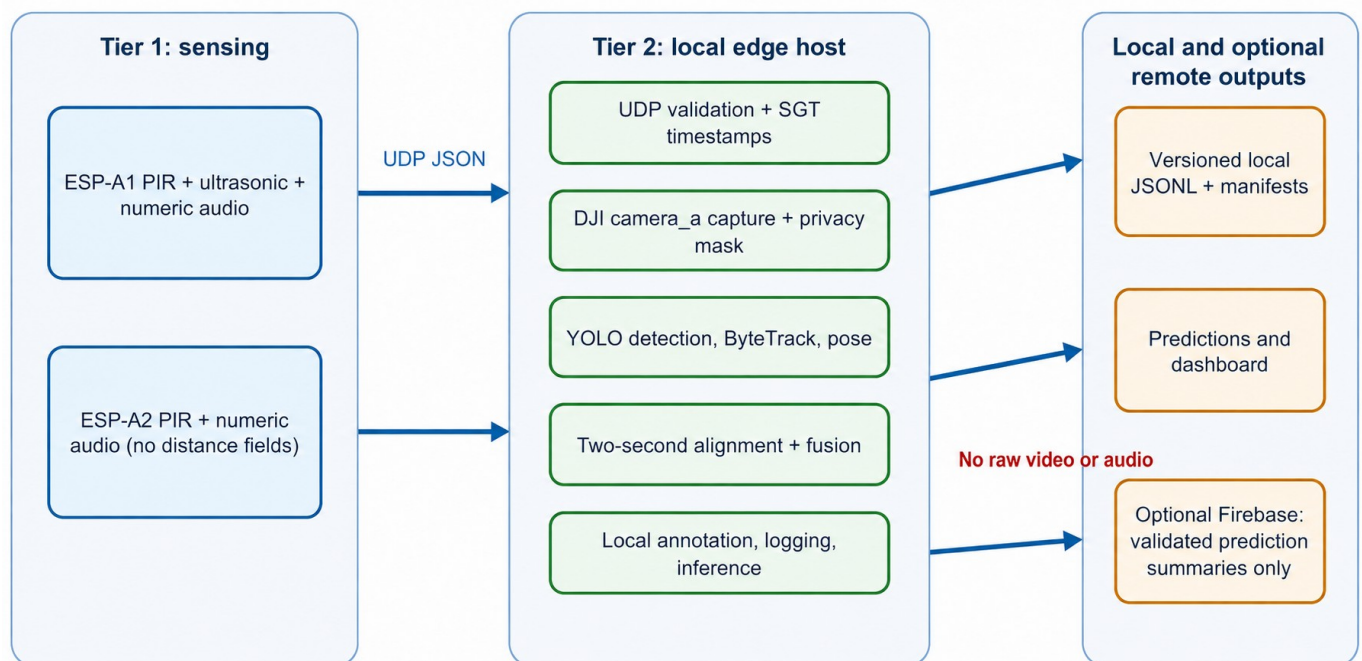


Figure 1: Active Zone A system architecture

Table 3: Active hardware and data streams

Component	Configuration	Produced evidence
ESP-A1	A_PIR_US_MIC; local UDP 4210	PIR, valid distance, numerical audio features
ESP-A2	A_PIR_MIC; local UDP 4211	PIR, numerical audio features
Laptop receiver	UDP 5005; ESP-A1/A2 only	Validated SGT receive envelopes and sequence monitoring
camera_a	DJI Osmo Action 5 Pro	Masked frames, detections, local tracks, pose, timing fields
Edge host	Python, OpenCV, YOLO, ByteTrack	Two-second feature windows, annotations, manifests, later predictions

Figure 1 groups the architecture into sensing, local edge processing, and local and optional remote outputs. Tier 1 performs low-cost numerical sensing, with each ESP board calculating audio-amplitude statistics locally and transmitting approximately one UDP packet per second. Tier 2 runs on the laptop and handles validation, privacy masking, detection, tracking, pose extraction, temporal alignment, annotation, and inference. Local logs and the dashboard remain on the laptop, while optional remote reporting through Firebase receives only validated prediction summaries. This division keeps identifiable media on the local machine and allows inference to continue when cloud connectivity is unavailable.

Each boundary has one responsibility. ESP nodes emit identified numerical measurements without classifying or retaining audio. The receiver validates identity, schema and timing; the vision subsystem accepts only masked frames; and fusion consumes validated numerical records rather than hardware or original video.

This separation also makes failures easier to interpret. Camera and sensor acquisition can degrade independently, and their availability is carried forward rather than hidden through default values. Versioned schemas link each record to its source, timestamp, configuration, and session, allowing the modelling data to be reconstructed without collecting the original sessions again. Firebase synchronisation remains outside the local collection, masking, and inference path.

3.3. Privacy-First Collection Design

The privacy boundary is enforced before any visual analysis begins. After capture, the system verifies the camera identity and resolution, applies the approved retained-region mask and exclusion polygons, and only then permits detection or display. The implementation begins with an all-black frame and copies across only the approved staff-side pixels. A missing configuration, unverified mask, incorrect camera or resolution, empty retained region, or invalid polygon stops processing. Downstream components receive only masked frames and therefore cannot request the original image through the normal application path.

Before collection begins, preflight checks the operator’s consent confirmation, camera mode, approved mask state, absence of camera audio, freshness of both ESP nodes and sufficient local storage. Data writers start only after these checks produce a valid session manifest. During collection, detected sensor or stream failures, an invalid mask state, or an operator-reported privacy incident pause the session and isolate the

affected two-second windows. Privacy-critical incidents also require verified deletion before collection resumes. These controls make the required recovery actions explicit and testable.

These rules apply to participant collection, where the approved multimodal configuration must remain valid. The deployed Camera-only inference mode has a different dependency: loss of an ESP node raises a health warning but does not remove its camera input. Camera loss or an unverified privacy mask still blocks activity output. Distinguishing collection from inference avoids treating a sensor warning as permission to collect outside the approved configuration.

Software checks enforce the recorded consent and mask-verification states and reject invalid configurations. The operator remains responsible for confirming consent, checking physical mask alignment and reporting non-participant entry or camera movement. Once a fault is recorded, the lifecycle prevents further collection until the required recovery checks pass.

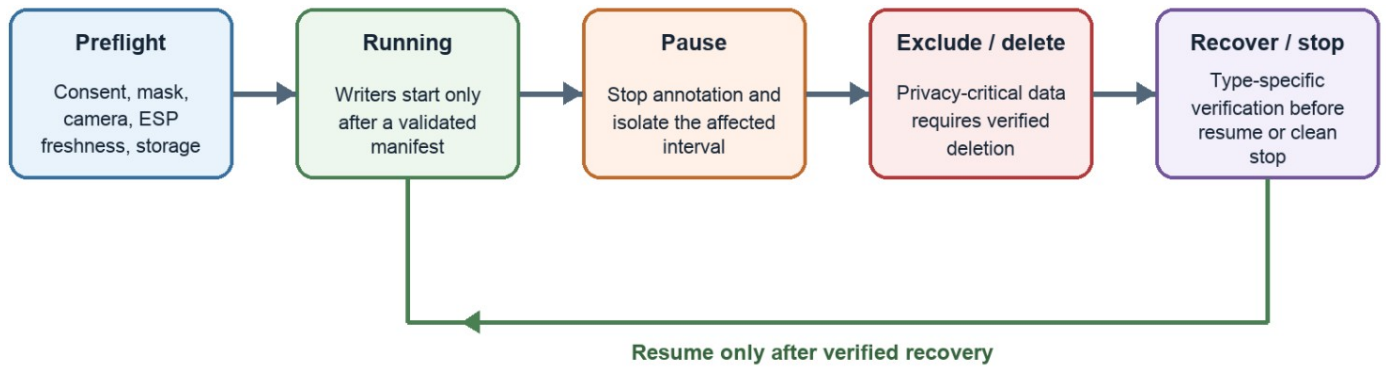


Figure 2: Fail-closed collection lifecycle

3.4. Temporal Alignment and Feature Design

Camera and ESP events use laptop receive timestamps normalised to Singapore Time (+08:00). The fusion layer floors each timestamp to a deterministic two-second boundary. Camera records, ESP-A1 packets and ESP-A2 packets are aggregated independently and then joined by window identifier. Rebuilding the same inputs produces the same windows. Incident-linked window identifiers can be excluded before feature computation, preventing deleted or invalid intervals from re-entering the modelling dataset. Host timestamps provide a common grouping reference, but differences in transmission and decoding delay can place measurements from the same physical event into different windows. This cross-modal timing offset was not independently quantified.



Figure 3: Deterministic alignment into a two-second multimodal window

Camera features include frame and track counts, pose availability, bounding-box centroids, detection confidence, keypoint confidence, joint-angle summaries and movement speed. Sensor features include packet counts, missing counts, PIR motion ratios, numerical audio summaries, RSSI and timing jitter. ESP-A1 additionally contributes valid ultrasonic distance summaries. Availability flags and missing counts remain explicit; missing modalities are never silently filled with values that could be mistaken for real inactivity.

The two-second window length balances temporal detail against sensor frequency and activity duration. Camera features may occur several times within one window, whereas each ESP node is expected to provide approximately two packets. A substantially shorter interval would produce many windows without sensor evidence, while a longer interval could combine brief Reaching/Handling movements with surrounding Serving/Processing or Idle/Waiting behaviour. Two seconds provides a consistent unit for annotation, missingness measurement and later classification without implying that all activities begin and end exactly on a boundary.

With identical validated records, configuration and exclusions, reconstruction produces identical identifiers and features. Annotation or incident corrections therefore do not require recollection, and all experimental views can derive from one frozen reconstruction.

Table 4: The 23 input fields used by the Camera-only classifier

Feature group	Fields	Meaning
Availability	3	Camera and pose flags, including the nested camera flag
Observation counts	9	Frames, records, tracks, gaps and available/missing pose counts
Pose coverage	1	Fraction of track records with an available pose
Position	2	Mean bounding-box centre x and y in pixels
Confidence	2	Mean detection and keypoint confidence
Pose and movement	2	Mean valid joint angle and centroid speed
Timing	4	Interval count, mean interval, interval spread and jitter

These 23 fields are the inputs before missing-value indicators are added. Some counts and protocol fields are constant after track filtering, so they are not 23 independent sources of discriminative information. Pose angles use confidence-qualified shoulder–elbow–wrist and hip–knee–ankle triplets. Movement is the distance between successive confident-keypoint centroids divided by elapsed time, averaged within a window. For illustration, a 50-pixel centroid displacement over 0.5 seconds gives 100 pixels per second.

Position and speed measured in pixels depend on camera placement and distance from the subject. Aggregation reduces a variable number of observations to a compact, fixed-length input, but also discards information. For example, joint-angle sequences (60°, 120°, 120° and 60°) and (90°, 90°, 90° and 90°) both average to 90° despite describing different movement patterns. Other features, such as movement speed, may distinguish these sequences, but the mean angle alone cannot preserve their temporal variation. Movement estimates also depend on which keypoints pass the confidence threshold. Changes in that set can shift its centroid without equivalent physical movement; the effect of this source of error was not separately measured.

3.5. Activity Taxonomy and Evaluation Design

Table 5: Activity taxonomy and expected evidence

Class	Operational interpretation	Potential discriminative evidence	Known ambiguity
Serving/Processing	Active service interaction/transaction processing	Forward pose, hand motion, nearby distance, audio activity	Serving and typing may share posture; ambient sound not person-specific
Idle/Waiting	Available but not actively serving/handling	Low movement, stable distance, lower event intensity	Short pauses inside another task can resemble idling
Reaching/Handling	Reach/manipulation of documents/equipment	Arm angles, wrist movements, brief proximity changes	Brief actions, occlusion, strong class imbalance

The final evaluation separates activity classification from person-association quality. Classification uses per-class precision, recall and F1, macro F1, balanced accuracy, confusion matrices and reliability measures. Tracking evidence is reported separately through controlled transitions, fragmentation, duplicate-ID observations, pose availability and track-presence audits; a complete deployment IDF1/HOTA benchmark was not produced and is retained as a limitation. Processing-time measurements are reported separately for vision, model inference and local delivery. Failure analysis combines controlled tracking observations, missing-input tests and aggregate classification errors; it does not provide condition-labelled error rates for each occlusion or posture scenario.

All candidate models were compared on the same 24,046 development rows using eight fixed leave-one-session-out folds, with preprocessing fitted separately inside each training fold. The 2,068-row external test isolated both its participants and its session and was opened only once, after Camera-only had been selected.

Because PIR, ultrasonic, and numerical microphone features describe Zone A rather than a unique individual, sensor-only scores are treated as ablation evidence rather than proof of person attribution. A

separate microphone-only model was not included in the retained comparisons, so the contribution of acoustic features cannot be isolated from PIR and ultrasonic measurements.

3.6. Operational Resilience and Data Integrity

The system lifecycle is designed around explicit states rather than assuming uninterrupted collection. Before a session enters the running state, preflight must confirm the active configuration, consent acknowledgements, camera identity and resolution, verified mask, absence of camera audio, freshness of ESP-A1 and ESP-A2, and sufficient local storage. A versioned session manifest is created before data writers begin. This establishes the intended participants, equipment and configuration against which later records can be checked.

Camera and sensor health are monitored separately from activity evidence. Loss of freshness, an invalid mask or a privacy incident pauses the session; affected records are counted but excluded. Observed, written and excluded totals expose the scale of failure, and resumption requires the recorded recovery action.

4. Implementation 2350/2500

4.1. ESP Telemetry and Ingestion

The two ESP8266 nodes share a common telemetry format but represent different physical configurations. Both read a PIR sensor and a MAX4466 microphone module, while ESP-A1 also reads an HC-SR04 ultrasonic sensor through the documented voltage divider. Every packet identifies the schema and firmware versions, node and sensor configuration, boot and sequence numbers, uptime, connectivity, RSSI, and numerical sensor values. ESP-A2 omits distance fields entirely, allowing downstream software to distinguish an absent sensor from an invalid measurement.

```
Navigation to Related Items et(data):
35 try:
36     payload = json.loads(data.decode("utf-8"))
37 except UnicodeDecodeError as error:
38     raise ValueError(f"Invalid UTF-8: {error}")
39 except json.JSONDecodeError as error:
40     raise ValueError(f"Invalid JSON: {error}")
41
42 if not isinstance(payload, dict):
43     raise ValueError("Packet must be a JSON object")
44
45 for field in REQUIRED_FIELDS:
46     if field not in payload:
47         raise ValueError(f"Missing field: {field}")
48
49 node_id = payload["node_id"]
50
51 if node_id not in EXPECTED_NODES:
52     raise ValueError(f"Unknown node_id: {node_id}")
53
54 if payload["zone"] != EXPECTED_NODES[node_id]:
55     raise ValueError(f"Wrong zone for {node_id}: expected {EXPECTED_NODES[node_id]}")
56
57 if not isinstance(payload["sequence"], int):
58     raise ValueError("sequence must be an integer")
59
60 schema_errors = sorted(PACKET_VALIDATOR.iter_errors(payload), key=lambda error: list(error.absolute_path),)
61
62 if schema_errors:
63     error = schema_errors[0]
64     field_path = ".".join(str(part) for part in error.absolute_path)
65     location = f" at {field_path}" if field_path else ""
66     raise ValueError(f"Packet schema validation failed{location}: {error.message}")
67
68 return payload
69
```

Figure 4: Active-node and schema validation in /src/aria/ingestion/packet_parser.py

The laptop UDP receiver binds to port 5005 and accepts only ESP-A1 and ESP-A2. Incoming bytes are decoded as UTF-8, parsed as JSON and checked against the telemetry schema. The parser verifies the node-to-zone mapping before the receiver writes a timestamped envelope. Invalid packets are not retained verbatim; the diagnostic contains only source metadata, size, error and a SHA-256 digest. Sequence and boot identifiers allow the node monitor to distinguish packet reordering from a genuine restart.

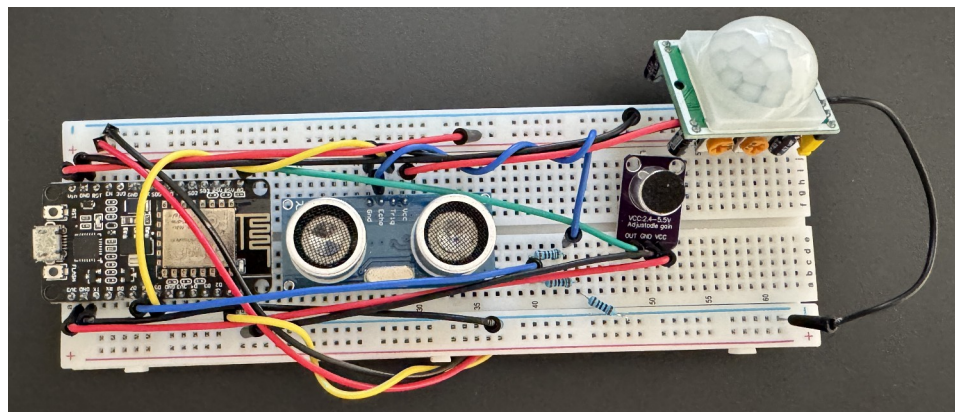


Figure 5: ESP-A1 with PIR, ultrasonic, and microphone sensors

4.2. Camera, Masking, and Vision Pipeline

The DJI camera publishes a wireless stream to a local MediaMTX bridge. A localhost FFmpeg relay copies the H.264 video, removes audio and exposes camera_a through RTSP. The camera manager accepts only the deployment source and verifies 1920x1080 at 30 FPS. This avoids built-in or Continuity camera fallback and ensures that a convenient but unapproved source cannot silently enter a participant session.

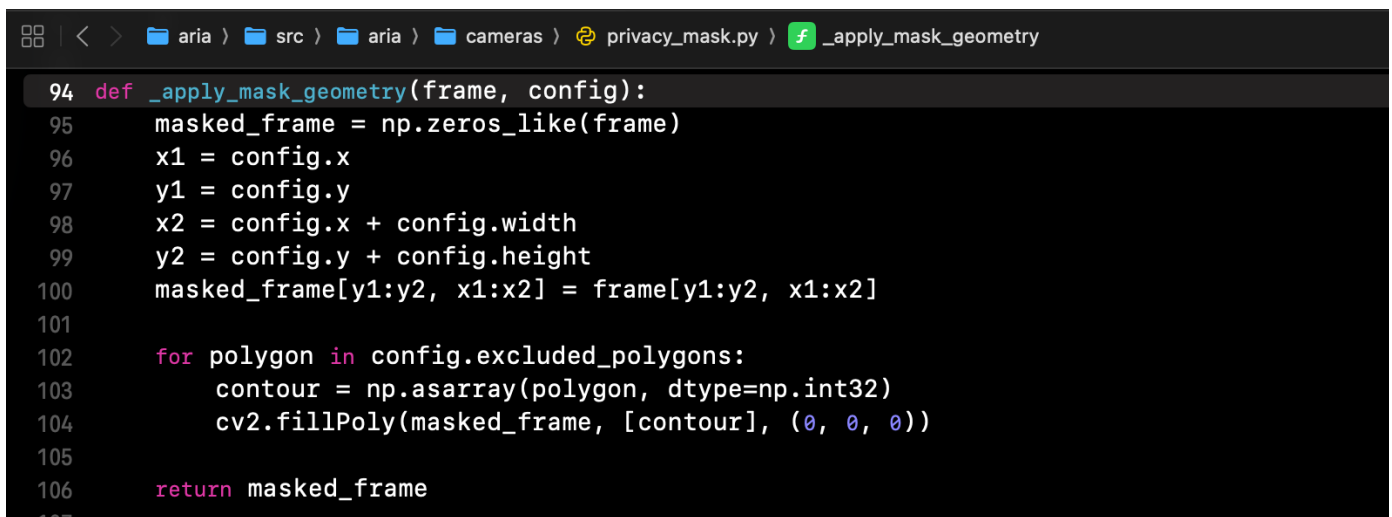
The retained configuration uses YOLOv8n detection and YOLOv8n-pose, with detector and pose input sizes of 768 and 384 pixels respectively. Detection confidence is 0.10, pose confidence 0.15, and a pose requires at least four keypoints at confidence 0.5.

```
ryanaw@MacBookAir aria % mediamtx config/mediamtx.local.yaml
2026/08/15 11:51:48 INF MediaMTX v1.19.3, darwin, arm64
2026/08/15 11:51:48 INF configuration loaded from
/Users/ryanaw/Desktop/Y3S2/CM3070FYP/aria/config/mediamtx.local.yaml
2026/08/15 11:51:48 INF [RTSP] started with listeners on 127.0.0.1:8554 (TCP/RTSP)
2026/08/15 11:51:48 INF [RTMP] started with listener on :1935 (TCP/RTMP)
```

Video capture runs in a dedicated thread that drains the decoder continuously, timestamps each frame, and applies the verified privacy mask immediately. The implementation retains only the newest waiting masked frame. When inference is slower than capture, an older waiting frame is replaced instead of allowing a stale queue to grow. Runtime summaries report captured, processed, replaced, and failed frames, making the trade-off between responsiveness and frame coverage visible.

The mask configuration contains a retained rectangle plus polygon exclusions. Geometry checks reject points outside the source frame, degenerate polygons, mismatched camera identifiers and resolution changes. The approved batamfast-v2 mask was calibrated after the final physical camera placement and reverified after deliberate movement. Development full-frame masks remain restricted to non-research test video.

Immediately after acquisition, the implementation builds a binary mask, creates a black frame and copies only approved pixels before detection, pose, display or feature extraction. A stream or mask failure marks the frame unavailable and surfaces the error; there is no unmasked fallback.



```
aria > src > aria > cameras > privacy_mask.py > _apply_mask_geometry
94 def _apply_mask_geometry(frame, config):
95     masked_frame = np.zeros_like(frame)
96     x1 = config.x
97     y1 = config.y
98     x2 = config.x + config.width
99     y2 = config.y + config.height
100     masked_frame[y1:y2, x1:x2] = frame[y1:y2, x1:x2]
101
102     for polygon in config.excluded_polygons:
103         contour = np.asarray(polygon, dtype=np.int32)
104         cv2.fillPoly(masked_frame, [contour], (0, 0, 0))
105
106     return masked_frame
```

Figure 6: Creation of masked frame in /src/aria/cameras/privacy_mask.py

4.3. Tracking, Pose, and Feature Records

After masking, YOLO detects people and ByteTrack assigns camera-local identities. A separate batched YOLO pose pass then estimates COCO-format keypoints for each accepted track. Keeping detection and pose separate means that a temporary pose failure does not delete or renumber an otherwise valid track. Each record retains the bounding box, confidence, availability, processing latency, and frame age. If too few keypoints meet the confidence threshold, pose is marked unavailable rather than replaced with plausible-looking coordinates.

Figure 7 shows the batched pose path used after tracking. Valid track crops are collected into one model call rather than inferred serially. Empty crops remain explicit empty outputs, the highest-confidence pose is selected when a crop contains more than one detection, and keypoints are translated back into masked-frame coordinates. This reduces repeated model overhead while keeping each pose tied to its camera-local box. Association with a track crop does not guarantee correct person association: during overlap, the highest-confidence pose may belong to a neighbouring person. This risk was not quantified separately from tracking and classification errors.

Within-frame duplicate suppression compares box overlap and shared confident keypoints conservatively, because an aggressive rule could remove a real adjacent person. A separate geometry-based stitcher was implemented experimentally, but no accepted end-to-end validation justified enabling it for participant collection. The tested three-frame new-ID confirmation gate was a different candidate and also failed continuity checks. Formal collection therefore retained raw camera-local IDs and required the operator to confirm replacement tracks.

Figure 8 shows selected branches of the experimental stitcher. A unique recent candidate permits association; multiple candidates or competing claims produce an uncertain result. The omitted candidate search also checks overlap, proximity, size and elapsed time. The extract illustrates a rejected-for-deployment option, not the tracking path used to generate the modelling data.

```
aria > src > aria > vision > pose_estimator.py > PoseEstimator

3 class PoseEstimator:
50 def estimate_many(self, frame, bounding_boxes):
51     outputs = [([], []) for _ in bounding_boxes]
52     crops = []
53     crop_metadata = []
54
55     for index, bounding_box in enumerate(bounding_boxes):
56         x1, y1, x2, y2 = bounding_box
57         crop = frame[y1:y2, x1:x2]
58         if crop.size == 0:
59             continue
60         crops.append(crop)
61         crop_metadata.append((index, x1, y1))
62
63     if not crops:
64         return outputs
65
66     options = {
67         "source": crops,
68         "conf": self.confidence,
69         "imgsz": self.inference_size,
70         "verbose": False,
71     }
72     if self.device:
73         options["device"] = self.device
74     results = self.model.predict(**options)
75
76     for result, (index, offset_x, offset_y) in zip(
77         results,
78         crop_metadata,
79     ):
80         if (
81             result.keypoints is None
82             or result.keypoints.xy is None
83             or len(result.keypoints.xy) == 0
84         ):
85             continue
86
87         pose_index = 0
88         if result.bboxes is not None and len(result.bboxes) > 1:
89             pose_index = int(result.bboxes.conf.argmax().item())
90
91         positions = result.keypoints.xy[pose_index].cpu().tolist()
92         if result.keypoints.conf is None:
93             confidences = [None] * len(positions)
94         else:
95             confidences = (
96                 result.keypoints.conf[pose_index].cpu().tolist()
97             )
98
99         outputs[index] = (
100             [
101                 [float(x + offset_x), float(y + offset_y)]
102                 for x, y in positions
103             ],
104             [
105                 None if value is None else float(value)
106                 for value in confidences
107             ],
108         )
109
110     return outputs
```

Figure 7: Batched pose estimation and coordinate restoration in `/src/aria/vision/pose_estimator.py`

```

25 class AnonymousTrackStitcher:
58     def associate(self, tracks, observed_at):
123         if len(candidates) == 1:
124             tentative_candidates[local_id] = candidates[0]
125         elif len(candidates) > 1:
126             associations[local_id] = _Association(
127                 None,
128                 "uncertain",
129                 "multiple_candidates",
130             )
131         else:
132             stitched_id = self._allocate_stitched_id()
133             associations[local_id] = _Association(
134                 stitched_id,
135                 "new",
136                 "new_track",
137             )
138             reserved_stitched_ids.add(stitched_id)
139
140             claims_by_stitched_id = {}
141             for local_id, state in tentative_candidates.items():
142                 claims_by_stitched_id.setdefault(
143                     state.stitched_track_id,
144                     [],
145                 ).append(local_id)
146             for stitched_id, claimants in claims_by_stitched_id.items():
147                 if len(claimants) > 1:
148                     for local_id in claimants:
149                         associations[local_id] = _Association(
150                             None,
151                             "uncertain",
152                             "multiple_candidates",
153                         )
154             continue

```

Figure 8: Experimental anonymous stitching branches (not deployed) in `/src/aria/vision/track_stitcher.py`

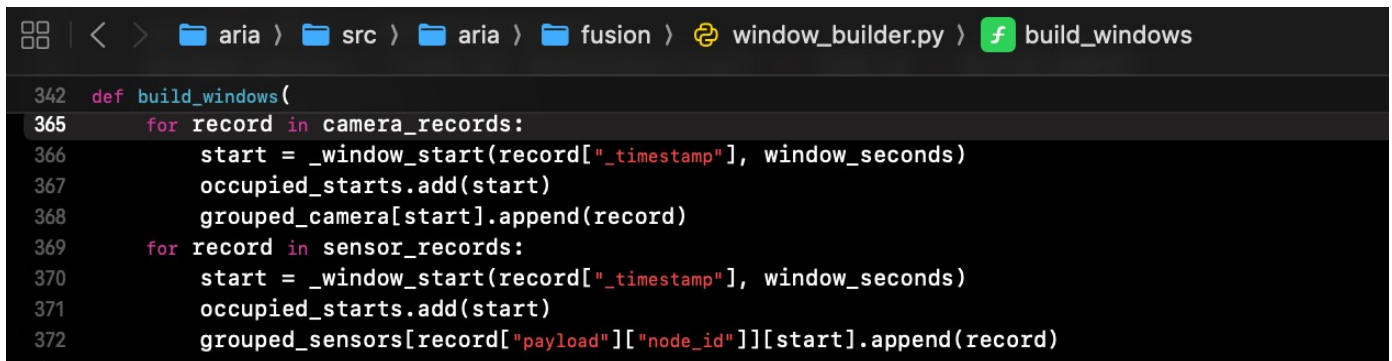
Tracking records retain timestamps, local IDs, boxes and pose availability. These support the controlled observations reported in Section 5.3, but do not by themselves establish ground-truth identity. No complete deployment IDF1 or HOTA evaluation was retained; independent reference associations would be required to compute those metrics credibly.

Table 6: Principal implementation modules

Module	Responsibility	Important design choice
packet_parser/udp_receiver	Validate and persist ESP packets	Active-node allow-list, privacy-safe invalid diagnostics
privacy_mask	Validate and apply retained-region geometry	Mask precedes detection, preview, and output
Vision subsystem	Latest-frame capture, detection, tracking, pose	Avoid stale queues; preserve explicit pose failure
fusion.window_builder	Build deterministic two-second features	SGT boundaries, missingness, exclusion-aware rebuild

Module	Responsibility	Important design choice
collection.preflight	Block unsafe startup	Manifest-before-writers and seven required checks
collection.lifecycle	Pause, exclude, delete, recover, stop	Writer-owned accounting, verified recovery
collection.live_annotation	Participant-card activity annotation	Window-aligned transitions and replacement-track prompt
dataset.participant_feature_builder	Build audited participant rows	Track-scoped camera features; shared sensor provenance
modeling	Train grouped Random Forest candidates	Fold-local preprocessing; external test remains locked
inference	Run the frozen Camera-only model	Hash verification, per-track isolation, unavailable state
dashboard/cloud	Present local state and project safe cloud summaries	No media or identity; durable immutable outbox

4.4. Multimodal Window Construction



```

342 def build_windows(
365     for record in camera_records:
366         start = _window_start(record["_timestamp"], window_seconds)
367         occupied_starts.add(start)
368         grouped_camera[start].append(record)
369     for record in sensor_records:
370         start = _window_start(record["_timestamp"], window_seconds)
371         occupied_starts.add(start)
372         grouped_sensors[record["payload"]["node_id"]][start].append(record)

```

Figure 9: Deterministic window grouping in `/src/aria/fusion/window_builder.py`

Simplified labelled-window example (illustrative values and abbreviated field names)

```

window_id: zone_a_<timestamp>_2000  local_track_id: 3
camera_available: true                pose_available: true
esp_a1_packets: 2                    esp_a1_available: true
esp_a2_packets: 1                    esp_a2_missing_count: 1
activity: Reaching/Handling           label_status: labelled

```

This simplified example distinguishes person-linked camera features from shared Zone A context; it is not a literal schema-valid record. The activity label and local track identifier are metadata and are not supplied as classifier input features.

Window construction assigns each valid camera or sensor record to a fixed two-second interval by rounding its host-aligned timestamp down to the interval boundary. Records are first aggregated independently by source and are then joined using the common session and window identifiers. This means that a missing ESP-A2 packet does not prevent camera and ESP-A1 evidence from contributing to the same window.

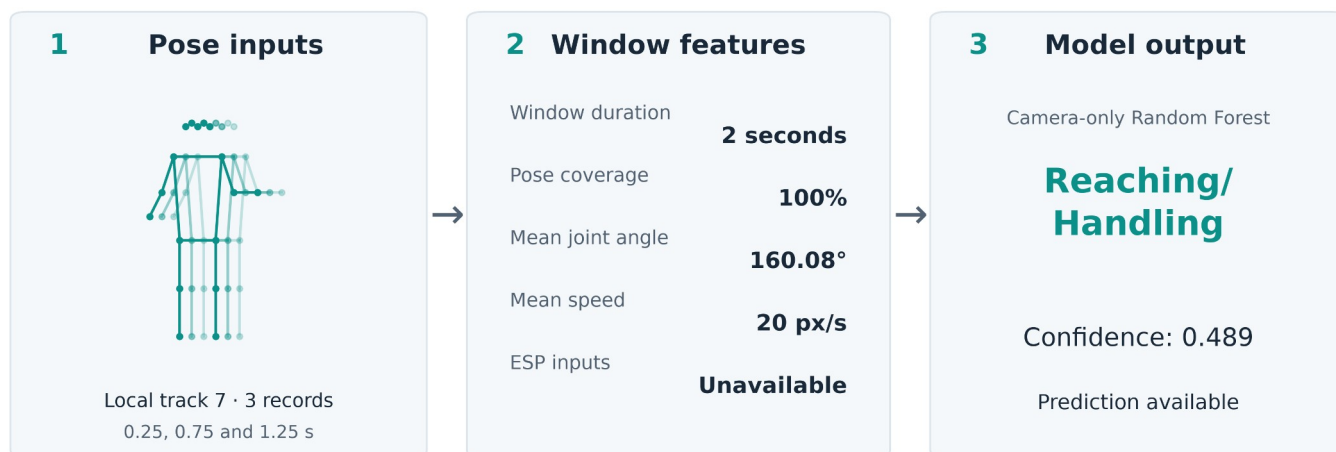
For every source, the window records availability, observation and rejection counts, and the features that were successfully computed. Missing evidence is represented by an availability flag rather than by zero, which may itself be a valid measurement. Excluded intervals are removed before feature generation, while deterministic sorting and source/configuration references allow the same model-ready data to be reproduced without retaining raw video or audio.

During reconstruction, each validated annotation is joined to the selected camera-local track within its two-second window, after which the shared Zone A sensor features are attached with their availability information. Rows with missing tracks, uncertain labels, explicit exclusions, or conflicting participant-track assignments are set aside rather than repaired by inferring identity. The resulting participant-feature dataset can be reproduced from the same sources, exclusions, and grouped assignments without repeating data collection.

Figure 10 follows a controlled numerical example through the implemented feature builder and selected model. Three pose records for local track 7 occur at 0.25, 0.75 and 1.25 seconds within one window. Translating every keypoint by 10 pixels between records gives a mean centroid speed of 20 pixels per second; the computed mean valid joint angle is 160.08 degrees. With no ESP packets, sensor availability remains false rather than being interpreted as inactivity.

From pose records to an activity prediction

Worked example using controlled numerical inputs



Each pose shifts 10 pixels every 0.5 seconds, giving a mean speed of 20 pixels per second.

Constructed inputs, not participant measurements. The output demonstrates execution, not recognition accuracy.

Figure 10: Worked feature-to-prediction example using controlled numerical pose inputs

The service returned Reaching/Handling with confidence 0.489 and a schema-valid prediction record. This demonstrates numerical aggregation, missing-input handling and execution of the frozen Camera-only model; the constructed pose has no ground-truth activity label and is not evidence of correct reaching recognition. Camera capture, masking and pose estimation are outside this particular check.

4.5. Session Lifecycle and Annotation

The live-session entry point combines preflight, manifest creation, telemetry and vision writers, incident handling and annotation. Each consenting participant has a persistent card linked to a visible camera-local track. The operator selects the dominant observable activity; the server supplies the two-second boundary

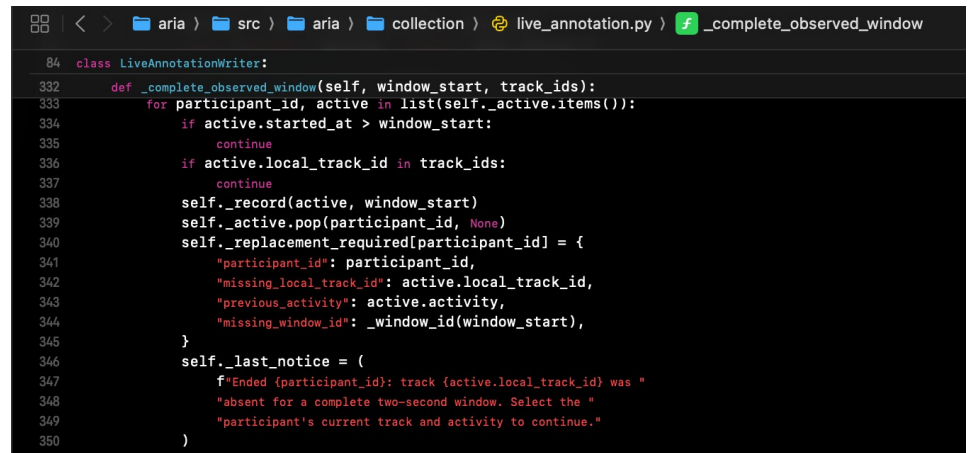
rather than requiring manual clock transcription. The manifest contains pseudonymous scope and configuration, not names or consent documents.

The annotation interface assigns activities to visible local tracks at two-second boundaries. Status is labelled, uncertain or excluded, with reasons covering ambiguous activity, occlusion, missing inputs and privacy incidents. Polling preserves unfinished input; track disappearance triggers a replacement prompt.

All 1,996 retained source annotations use one annotator code. Concurrent staff were labelled through separate participant cards, and intervals were split when activity, track or validity changed. When no dominant class was defensible, the interval remained uncertain and was excluded from modelling. There is no retained independent double-annotation study or inter-rater agreement estimate. Timestamp

automation reduces transcription burden, but does not remove observer judgement, reaction delay or ambiguity between a short pause and Idle/Waiting.

Figure 11 shows the fail-closed response when a selected track is absent for a complete window: the annotation closes and explicit reassignment is required. The previous activity remains in the prompt, preventing silent transfer to an unverified track.



```
84 class LiveAnnotationWriter:
332     def _complete_observed_window(self, window_start, track_ids):
333         for participant_id, active in list(self._active.items()):
334             if active.started_at > window_start:
335                 continue
336             if active.local_track_id in track_ids:
337                 continue
338             self._record(active, window_start)
339             self._active.pop(participant_id, None)
340             self._replacement_required[participant_id] = {
341                 "participant_id": participant_id,
342                 "missing_local_track_id": active.local_track_id,
343                 "previous_activity": active.activity,
344                 "missing_window_id": _window_id(window_start),
345             }
346             self._last_notice = (
347                 f"Ended {participant_id}: track {active.local_track_id} was "
348                 "absent for a complete two-second window. Select the "
349                 "participant's current track and activity to continue."
350             )
```

Figure 11: Fail-closed handling of a missing camera-local track in `/src/aria/collection/live_annotation.py`

4.6. Audited Dataset and Frozen Model

Reconstruction expanded 1,996 source intervals into 28,139 participant-window annotations without changing the originals. Quality exclusions left 26,114 feature rows, 13,112 unique session-time windows and 1,847 contiguous activity episodes. The audit found 461 conflicting window-track groups affecting 922 rows; 865 were otherwise usable and became additional exclusions. This distinction explains why conflict groups and excluded rows have different totals. Camera statistics use only the selected local track; ESP statistics remain shared Zone A context. An episode is a consecutive sequence of retained windows within one session with the same participant, camera-local track and activity label. Track changes or excluded gaps can therefore split a single real-world task into several dataset episodes.

Eight sessions supplied development data; one separate participant-session component supplied the held-out test. For every modality subset, three Random Forest configurations were compared: 100 trees/depth 12/minimum leaf 1; 100 trees/unrestricted depth/minimum leaf 2; and 200 trees/depth 18/minimum leaf 2. All used balanced-subsample class weighting and seed 3070. Selection maximised pooled development macro F1, with balanced accuracy as the first tie-breaker. Camera-only selected the first configuration. It was refitted on all development rows, then evaluated once without test-driven tuning or calibration.

Figure 12 shows the training boundary. A fresh estimator is created inside each held-out-session fold, so median imputation and missing-value indicators are fitted using training rows only. Null features become NaN before fitting; all-missing columns are retained consistently. The fitted transform and forest are reused

for inference. Availability and missingness indicators distinguish absent evidence from real zero measurements, although they may also encode session-specific operating conditions.

```
aria > src > aria > modeling > baselines.py > No Selection
268 def _rf_pipeline(candidate: dict, seed: int) -> Pipeline:
269     return Pipeline([
270         ("imputer", SimpleImputer(
271             strategy="median",
272             add_indicator=True,
273             keep_empty_features=True,
274         )),
275         ("classifier", RandomForestClassifier(
276             random_state=seed,
277             n_jobs=1,
278             **candidate,
279         )),
280     ])
281
282 def _cross_validated_predictions(
283     matrix: np.ndarray,
284     labels: list[str],
285     row_folds: list[str],
286     folds: list[str],
287     estimator_factory,
288 ) -> tuple[list[str], list[dict]]:
289     predictions = [None] * len(labels)
290     fold_reports = []
291     for fold_id in folds:
292         training = np.asarray([value != fold_id for value in row_folds])
293         validation = ~training
294         estimator = estimator_factory()
295         estimator.fit(matrix[training], np.asarray(labels)[training])
296         fold_predictions = estimator.predict(matrix[validation]).tolist()
297         validation_indexes = np.flatnonzero(validation).tolist()
298         for index, prediction in zip(validation_indexes, fold_predictions):
299             predictions[index] = prediction
300         fold_reports.append({
301             "fold_id": fold_id,
302             "training_row_count": int(training.sum()),
303             "validation_row_count": int(validation.sum()),
304             "metrics": _metrics(np.asarray(labels)[validation].tolist(), fold_predictions),
305         })
306     if any(value is None for value in predictions):
307         raise BaselineBuildError("cross-validation did not predict every development row")
308     return predictions, fold_reports
309
310 def _candidate_sort_key(result: dict) -> tuple:
311     params = result["parameters"]
312     return (
313         -result["aggregate_metrics"]["macro_f1"],
314         -result["aggregate_metrics"]["balanced_accuracy"],
315         params["n_estimators"],
316         -1 if params["max_depth"] is None else params["max_depth"],
```

Figure 12: Fold-local preprocessing and Random Forest fitting in `/src/aria/modeling/baselines.py`

4.7. Local Inference, Dashboard, and Firebase Projection

The local Camera-only service verifies its model, feature contract and schema at startup, then reconstructs each track's feature order. Completed-window predictions are logged locally. Camera or model unavailability clears the previous activity rather than displaying a stale result.

The localhost dashboard retains only the latest completed state: camera-local activity, confidence, camera and mask status, ESP freshness and fixed warnings. It exposes no video, participant/session identifiers, arbitrary diagnostics or raw features. An unverified mask blocks activity display, including earlier predictions, as shown below.

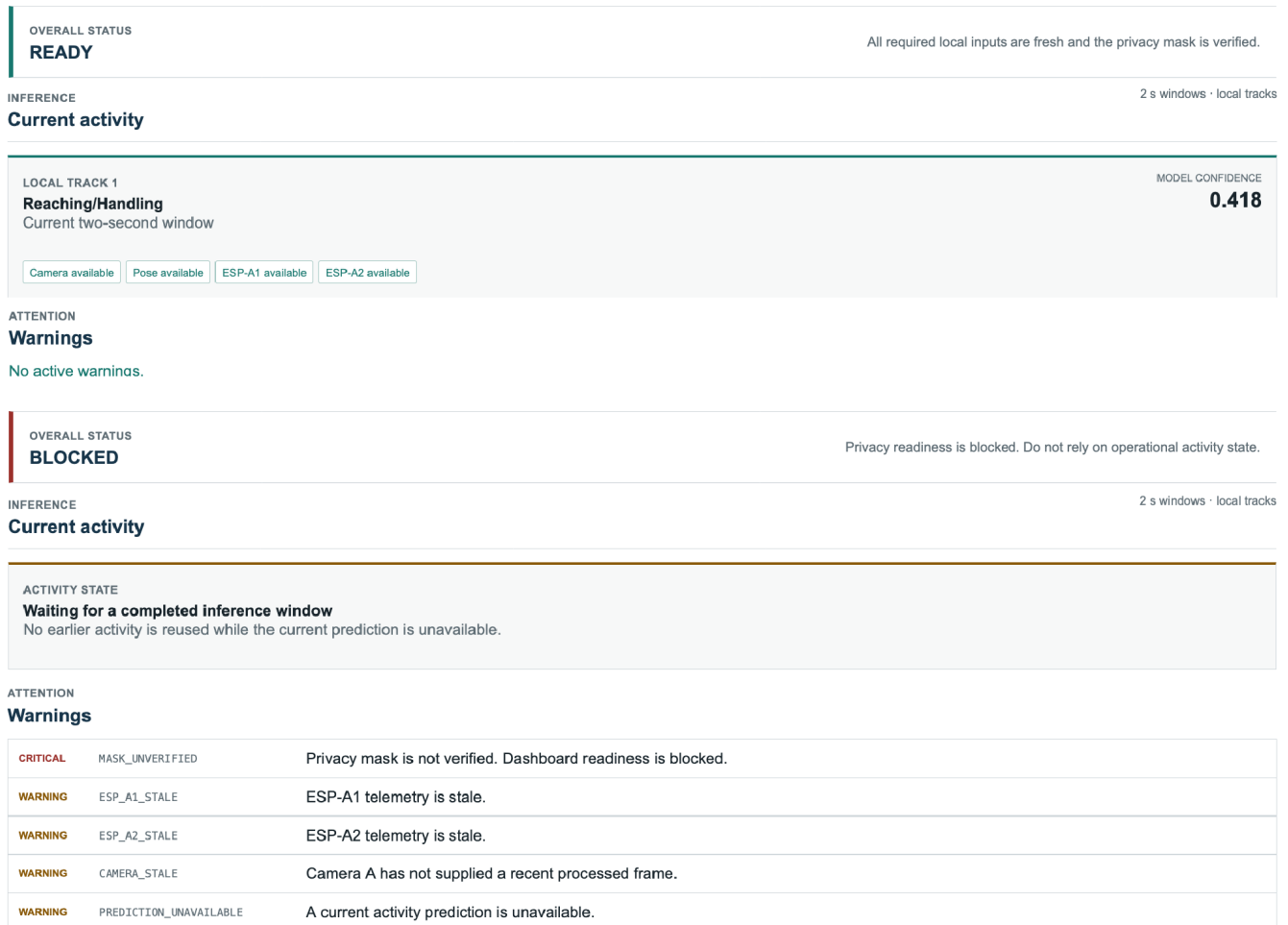


Figure 13: Dashboard extracts during controlled tests: ready state with activity displayed (top), and blocked state with activity cleared and privacy and stale-input warnings visible (bottom)

Figure 14 shows how inference fails safely. If the model raises an exception, the service returns an unavailable result with no activity or confidence. The final record also carries the availability of each input and must pass the prediction schema before it is returned. This prevents an earlier successful prediction from being shown as current evidence.

```

61 class LocalInferenceService:
186     def predict(self, *, session_id, local_track_id, feature_vector, emitted_at=None):
214         except Exception:
215             result = {
216                 "prediction_available": False,
217                 "predicted_activity": None,
218                 "confidence": None,
219                 "unavailable_reason": "model_error",
220             }
221
222     window_id = feature_vector["window_id"]
223     identity = (
224         f"{session_id}|{window_id}|{local_track_id}|"
225         f"{self.config['model']['version']}|"
226         f"{self.config['preprocessing']['version']}"
227     )
228     record = {
229         "schema_version": self.config["preprocessing"]["prediction_schema_version"],
230         "prediction_id": "prediction_" + sha256(identity.encode("utf-8")).hexdigest()[:24],
231         "session_id": session_id,
232         "window_id": window_id,
233         "zone": "A",
234         "local_track_id": local_track_id,
235         "emitted_at_sgt": _sgt_text(emitted_at),
236         "model_version": self.config["model"]["version"],
237         "preprocessing_version": self.config["preprocessing"]["version"],
238         "feature_schema_version": self.config["preprocessing"]["feature_schema_version"],
239         "prediction_available": result["prediction_available"],
240         "predicted_activity": result["predicted_activity"],
241         "confidence": result["confidence"],
242         "unavailable_reason": result["unavailable_reason"],
243         "availability": {
244             "camera_available": feature_vector["camera_available"],
245             "pose_available": feature_vector["pose_available"],
246             "esp_a1_available": feature_vector["a1_available"],
247             "esp_a2_available": feature_vector["a2_available"],
248         },
249     }
250     try:
251         self.prediction_validator.validate(record)
252     except ValidationError as error:
253         raise InferenceInputError(f"prediction schema failure: {error.message}") from error
254     return record
255

```

Figure 14: Fail-closed prediction output and schema validation in `/src/aria/inference/service.py`

Firebase is used only for optional remote reporting. Activity recognition and the dashboard run on the laptop and do not depend on Firebase delivery. A Firebase connection failure therefore does not interrupt local inference or dashboard operation. Only validated prediction summaries are placed in the upload queue; failed deliveries remain in a local outbox and are retried later. Video, audio, raw features, participant identifiers, and session identifiers are never sent to Firebase.

```

74 class FirebaseContract:
133 def project(self, prediction, *, uploaded_at=None):
134     """reduce one local prediction to the exact non-identifying cloud shape"""
135     try:
136         self.source_validator.validate(prediction)
137     except ValidationError as error:
138         raise FirebasePayloadError("source prediction does not satisfy the frozen schema") from error
139
140     payload = {
141         "schema_version": self.config["inputs"]["firebase_prediction_schema_version"],
142         "prediction_id": prediction["prediction_id"],
143         "emitted_at_sgt": prediction["emitted_at_sgt"],
144         "uploaded_at_sgt": _timestamp(uploaded_at),
145         "zone": prediction["zone"],
146         "model_version": prediction["model_version"],
147         "preprocessing_version": prediction["preprocessing_version"],
148         "prediction_available": prediction["prediction_available"],
149         "availability": dict(prediction["availability"]),
150     }
151     if prediction["prediction_available"]:
152         payload["predicted_activity"] = prediction["predicted_activity"]
153         payload["confidence"] = prediction["confidence"]
154     else:
155         payload["unavailable_reason"] = prediction["unavailable_reason"]
156     try:
157         self.cloud_validator.validate(payload)
158     except ValidationError as error:
159         raise FirebasePayloadError("projected prediction does not satisfy the Firebase schema") from error
160     if set(payload) - set(self.config["payload"]["allowed_fields"]):
161         raise FirebasePayloadError("projected prediction contains a forbidden field")
162     return payload

```

Figure 15: Privacy-bounded Firebase payload projection in `/src/aria/cloud/firebase_sync.py`

Figure 15 shows how the Firebase boundary is enforced in code. The projector builds a new payload from a limited set of summary fields, validates it against the Firebase schema and rejects any field outside the allowlist. Participant and session identifiers, raw features and media are therefore absent from the cloud record by construction.

Restoring privacy readiness does not restore an old prediction



A fresh prediction is required before activity can be displayed again.

Controlled software test: verification flags and fresh records were supplied by the test harness.

Figure 16: Observed dashboard states during a controlled mask-failure and recovery check

Figure 16 tests the dashboard state transition, complementing the ready and blocked screenshots. Starting with one valid test prediction, changing mask verification to false clears the prediction and camera state. Restoring verification alone leaves zero predictions and a degraded state. Only after fresh camera status and a new prediction are supplied does the dashboard return to ready.

This distinction prevents a pre-fault activity from reappearing as current evidence when privacy readiness returns. The check used the existing dashboard state and rendering functions with controlled records and verification flags; it did not involve participants, live mask validation or a measurement of hardware recovery time.

4.8. Frozen Release and Reproducibility

The final software was packaged as a versioned source archive with a SHA-256 manifest. It includes the active Zone A code, schemas, configuration, ESP-A1/A2 firmware, selected Camera-only model and required pose weights. Participant data, previous outputs, caches, retired firmware, alternative models and media files are excluded. The archive was extracted into a clean directory, its files were verified against the manifest, and the controlled numerical demonstration was rerun using the existing Python environment. This verifies the packaged software and demonstration, rather than a fresh dependency installation or independent reproduction of participant-based results.

Rebuilding the participant-based results requires the restricted numerical records. The exact files for two historical collection-profile variants could not be recovered, although their recorded hashes and contemporaneous configuration documentation remain available. This limits exact reproduction of the original acquisition setup.

4.9. Implementation Testing and Technical Challenges

Annotation input exposed a specific implementation fault: one-second page polling could reset an unfinished track value. Inspection traced this to rebuilding participant cards before the input was committed. The revised interface saved drafts on input events and preserved focused fields. A controlled browser test then entered a track, waited through multiple polls and selected an activity; the value, focus and highlight persisted. This verified the interface fix, not automatic identity recovery, which still required operator confirmation.

Shared sensor attribution remained unresolved: ambient measurements could describe several staff members. Track-specific camera features and shared sensor context were compared on identical development folds. Camera-only achieved 0.443 macro F1 versus 0.406 for the strongest fusion. Selecting Camera-only resolved deployment model selection, not person-specific acoustic attribution; fusion improvement remained unsupported.

Network loss required separate responses for cloud delivery and local sensing. The Firebase design used an isolated callback and durable retry queue; failure-injection and retry tests verified continued local inference, queued delivery and conflict rejection. This was a verified software response to cloud failure. For ESP delivery loss, explicit missingness and health warnings mitigated misleading output but did not recover lost measurements. The 90-minute run still recorded approximately 91.7% and 91.5% delivery, leaving network reliability unresolved.

5. Evaluation 1870/2500

5.1. Evaluation Strategy and Justification

The evaluation combines four forms of evidence. Data from the nine retained participant sessions were used to reconstruct the dataset, compare modality combinations and evaluate the selected model. All model comparisons used grouped development folds, followed by one evaluation of the selected model on the isolated participant-session test. Component and live-hardware tests measured sensor delivery, camera throughput, pose availability, latency and recovery. Dashboard, privacy-state and Firebase failure-handling tests used controlled non-participant records so that mask failure, stale sensors, duplicate delivery and network loss could be reproduced safely and consistently. Dataset audits checked source reconciliation, identity conflicts, missingness, privacy and split leakage.

Macro F1 was used as the primary model-selection measure to give each activity class equal weight. Reaching/Handling comprises only 5.29% of the development rows, so overall accuracy could favour the majority Serving/Processing class. Balanced accuracy and per-class precision, recall, and F1 reveal class-specific behaviour, while the confusion matrix shows which activities are confused. Leave-one-session-out development folds reduce temporal and session leakage. The external test also isolates participants, but because it contains only one session, it measures one difficult generalisation boundary rather than population-level performance.

5.2. Technical Evaluation and Testing Types

Technical evaluation used complementary testing levels. Unit and component tests checked packet validation, feature construction and prediction-schema rules in isolation. Integration tests checked agreement between offline and live feature ordering, missing-value handling, per-track isolation and local prediction delivery. These checks establish software consistency rather than recognition accuracy.

Functional and privacy tests exercised dashboard states, mask blocking, missing inputs and forbidden output fields. Fault-injection and recovery tests supplied controlled failures, repeated deliveries and conflicting records to check fail-closed behaviour and Firebase retry handling. Browser checks verified visible behaviour; database-rules emulator tests and a guarded live write/read/delete check tested the cloud boundary.

Performance and stability tests measured processing rates, latency and packet delivery over time (Section 5.3). Release verification checked archive integrity and the controlled demonstration after clean extraction. Separately, grouped development ablation and the one-time held-out evaluation measured classification performance (Sections 5.5 and 5.6). Table 7 summarises the retained outcomes; these testing types are complementary and their counts should not be added as independent experiments.

Table 7: Final verification evidence

Boundary	Evidence	Outcome
Dataset and splits	26,114 rows; schema, exclusion and leakage audits	PASS
Development modelling	24,046 out-of-fold predictions per candidate; deterministic rebuild	COMPLETE

Boundary	Evidence	Outcome
Dashboard	14 scenarios, 8 privacy checks, 3 rendered safety states	PASS
Firebase	20 focused checks plus 10 rules-emulator scenarios and guarded live write/read/delete	PASS
Functional and release assessment	121 focused release and assessment tests; fresh-extraction synthetic run	PASS
Final stability assessment	5,400.008 s; all metrics present; sustained degradation retained	Completed; delivery target not met

Functional, privacy and release checks passed within their tested scope, whereas the stability run did not meet the delivery target. An unverified mask cleared displayed activity, forbidden cloud fields were rejected and simulated Firebase delivery failure left local inference running. These are controlled software outcomes, not participant measurements or proof that all operational failure conditions were covered.

5.3. Sensor, Camera, and Runtime Reliability

Table 8: Selected live reliability and runtime results

Test	Metric	Result	Interpretation
Masked CV acceptance	Vision processing rate / processing-time p95	6.969 FPS / 164.149 ms	Feature output available; below 15 FPS target
Masked CV acceptance	Pose availability	2,876 / 2,977 (96.60%)	High coverage with occlusion failures
Earlier endurance test	ESP-A1 delivery	5,302 / 5,400 (98.185%)	Passed unchanged 98% target
Earlier endurance test	ESP-A2 delivery	5,290 / 5,400 (97.963%)	Two packets below target
Final stability test	Capture / processing	29.972 / 7.544 FPS	Capture outpaced processing; waiting frames were replaced
Final stability test	Completed-window processing p95	113.274 ms	Below two-second window duration
Final stability test	A1 / A2 estimated delivery	91.70% / 91.50%	Sustained degraded health retained

The final 90-minute run was a non-research live-display test with staff-record storage and cloud delivery disabled; only aggregate diagnostics were retained. It lasted 5,400.008 seconds and ended cleanly without measurement pauses. The longest observed degraded interval lasted 5,350.444 seconds, approximately 99.08% of the run. Public Wi-Fi may have contributed, but its causal contribution was not isolated. Continued inference demonstrates tolerance of sensor-delivery loss, not successful network delivery. The 15 FPS selection target and 98% per-node packet-delivery target were engineering acceptance thresholds, not empirically established minima for activity accuracy. At one expected packet per second, 98% permits 108 losses in 90 minutes; at 15 FPS a two-second window can contain about 30 processed frames. The observed

roughly 7.5 FPS gives fewer opportunities to capture brief actions, but the effect on classification was not separately measured. Packet-delivery percentages were estimated from observed forward sequence gaps, rather than an independent count of every transmitted packet. The 113.274 ms p95 covers completed-window processing across 2,700 samples, not the delay from a physical action to its appearance on the dashboard.

Table 9: Controlled tracking observations and limits

Controlled evidence	Observed outcome	Interpretation
31 July; 120.108 seconds	837 frames; 120 implausible poses rejected; 367 duplicate tracks suppressed	Filter operations, not verified counts of identity errors prevented
1 August; 5 trial rows across 4 scenarios	0 unexpected ID transitions; 0 behaviour failures	Stable sampled observations; exit/re-entry intentionally received a new ID
3 August recheck; 5 trial rows	3 unexpected transitions; 1 failed continuity row; 0 collision flags	Close side-by-side work exposed fragmentation
3 August confirmation-gate candidate; 4 trial rows	4 unexpected transitions; 2 failed rows	Separate three-frame confirmation experiment; not deployed

A trial row records one subject's observed ID sequence, not a frame-level identity annotation. The five-row checks each cover continuous presence, short occlusion, exit/re-entry and two-person separation. Full trial durations and exhaustive reference tracks were not retained for those checks, so switch rates per minute, IDF1 and HOTA are not inferred. The 120-second run is a separate throughput and pose test. Stable early observations did not generalise to every later multi-person condition; the report therefore retains tracking stability as only partially achieved.

5.4. Audited Dataset and Split Integrity

Table 10: Final audited modelling dataset

Measure	Result
Retained sessions / participants	9 / 9 pseudonymised Counter Agents
Normalised annotation rows	28,139
Included participant-feature rows	26,114
Serving/Processing	16,513 (63.23%)
Idle/Waiting	8,309 (31.82%)
Reaching/Handling	1,292 (4.95%)
Development / external	24,046 / 2,068 rows
Exclusions	1,145 uncertain; 2 source-excluded; 13 track-absent; 865 conflicting assignments

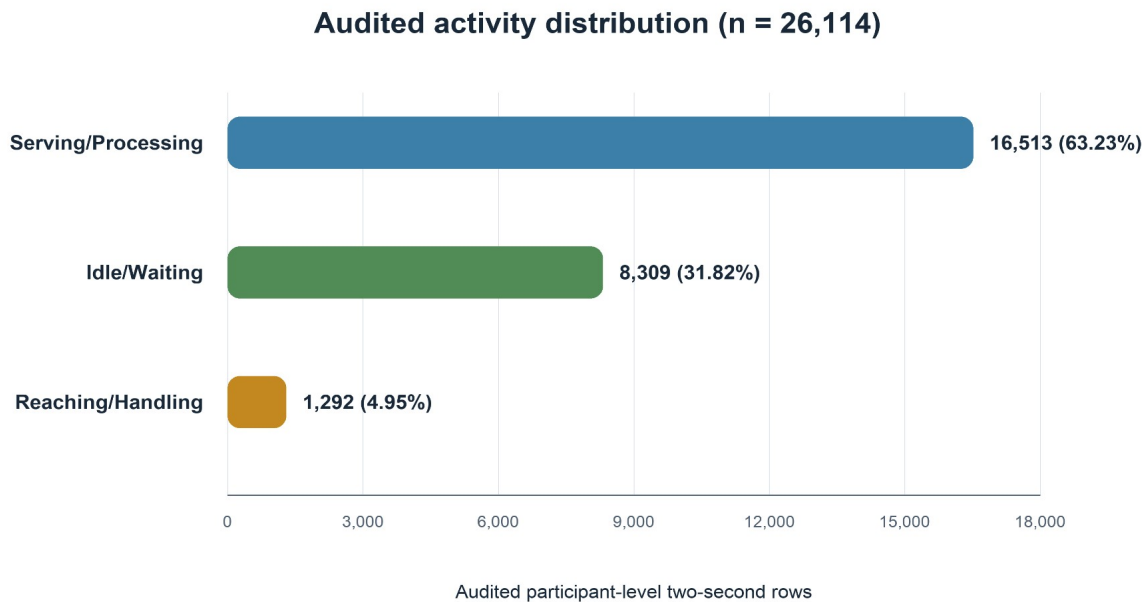


Figure 17: Audited activity distribution after quality exclusions

All included rows passed the schema and exclusion checks. Camera tracks are present in 100% of included rows, pose in 99.76%, ESP-A1 in 99.46%, and ESP-A2 in 97.86%. These are post-filtering availability rates, not unbiased measurements of capture or tracking accuracy: track-absent and conflicting assignments were removed. Individual contributions range from 389 to 5,959 rows, and adjacent windows remain correlated. A participant-window row describes one person for two seconds; simultaneous participants can therefore produce several rows for the same clock interval. The 26,114 rows represent 14 hours 30 minutes 28 seconds of summed participant time, not additional wall-clock collection beyond the nine sessions.

Quality exclusions removed 2,025 of 28,139 normalised rows (7.20%). Classification scores therefore describe retained, usable participant-track windows rather than every interval encountered by an unattended system. Operator-confirmed assignments and conflict exclusions also mean that these scores are not a joint measure of fully automatic identity tracking and activity recognition.

The participant-session graph contained only two disconnected components, so three non-empty partitions with both participant and session isolation were not possible. Development contains six participants across eight sessions; the held-out component contains three different participants in one session (Figure 18). Each development fold holds out a whole session, but its participants may also appear in training. Development therefore measures new-session performance with potentially familiar people. The final test measures one unseen-session/unseen-participant boundary at the same site, not an external-site benchmark.

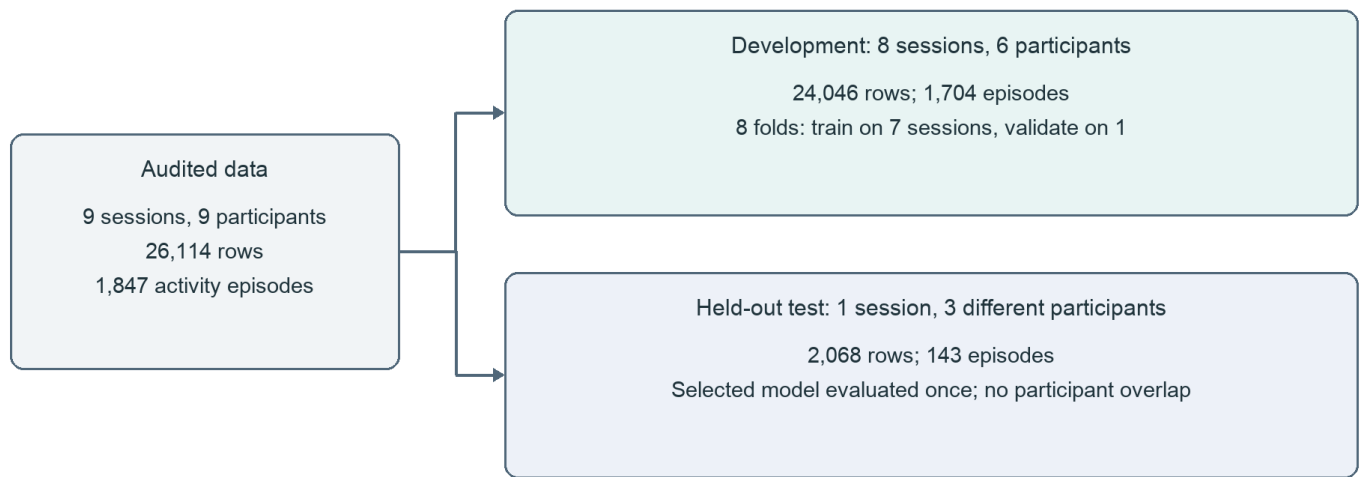


Figure 18: Session and participant separation in development and the held-out test

Figure 19 compares class proportions within the two partitions. Development contains 15,420 Serving/Processing, 7,353 Idle/Waiting and 1,273 Reaching/Handling rows; the test contains 1,093, 956 and 19 respectively. Reaching/Handling falls from 5.29% to 0.92%, while Idle/Waiting increases from 30.58% to 46.23%. Percentages, rather than raw counts alone, make this change visible despite the different partition sizes.

Class balance in development and the held-out test

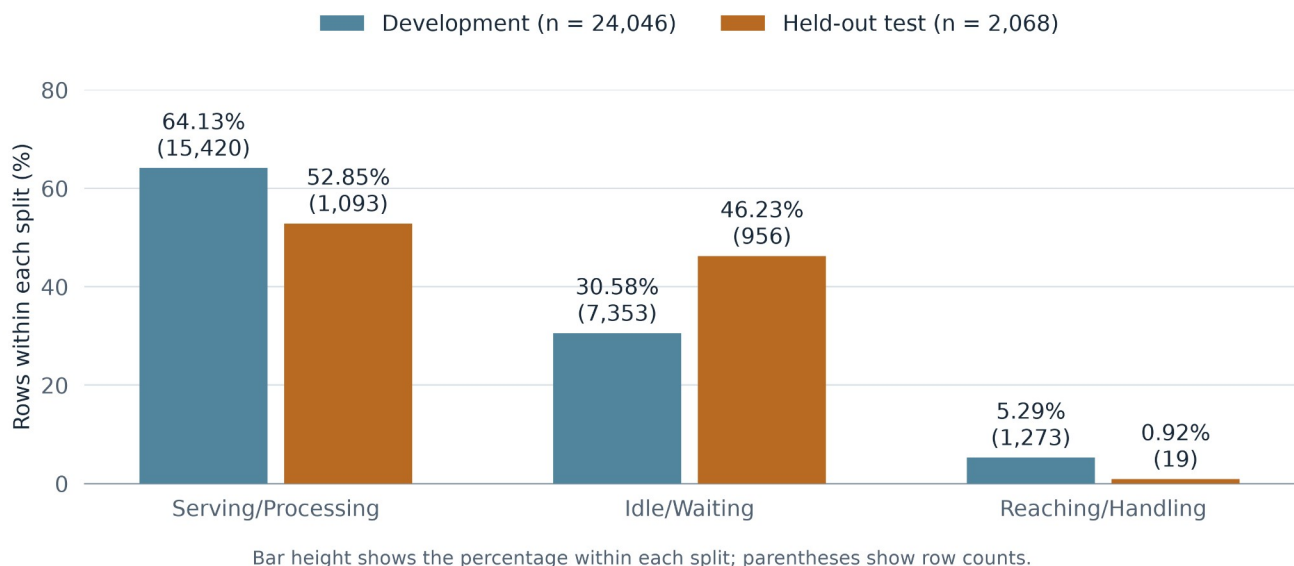


Figure 19: Class proportions in development and the held-out test; labels include row counts

The held-out boundary changes participants, session conditions and class balance together. The decline in macro F1 therefore cannot be attributed solely to unfamiliar participants. Macro F1 gives each class equal weight, but its precision terms still depend on the distribution of predictions and true classes. These percentages are calculated from the retained class supports, without rerunning the test or changing model selection.

5.5. Development Ablation and Model Selection

Table 11: Dev-only modality ablation across 8 frozen LOSO folds. Differences are calculated before rounding.

Candidate	Development macro F1	Difference from Camera-only
Majority baseline	0.260	-0.182
ESP-A1 only	0.369	-0.074
ESP-A1 + ESP-A2	0.370	-0.073
Camera A only	0.443	0.000
Camera + ESP-A1	0.406	-0.037
Camera + ESP-A2	0.405	-0.038
Camera + ESP-A1 + ESP-A2	0.391	-0.052

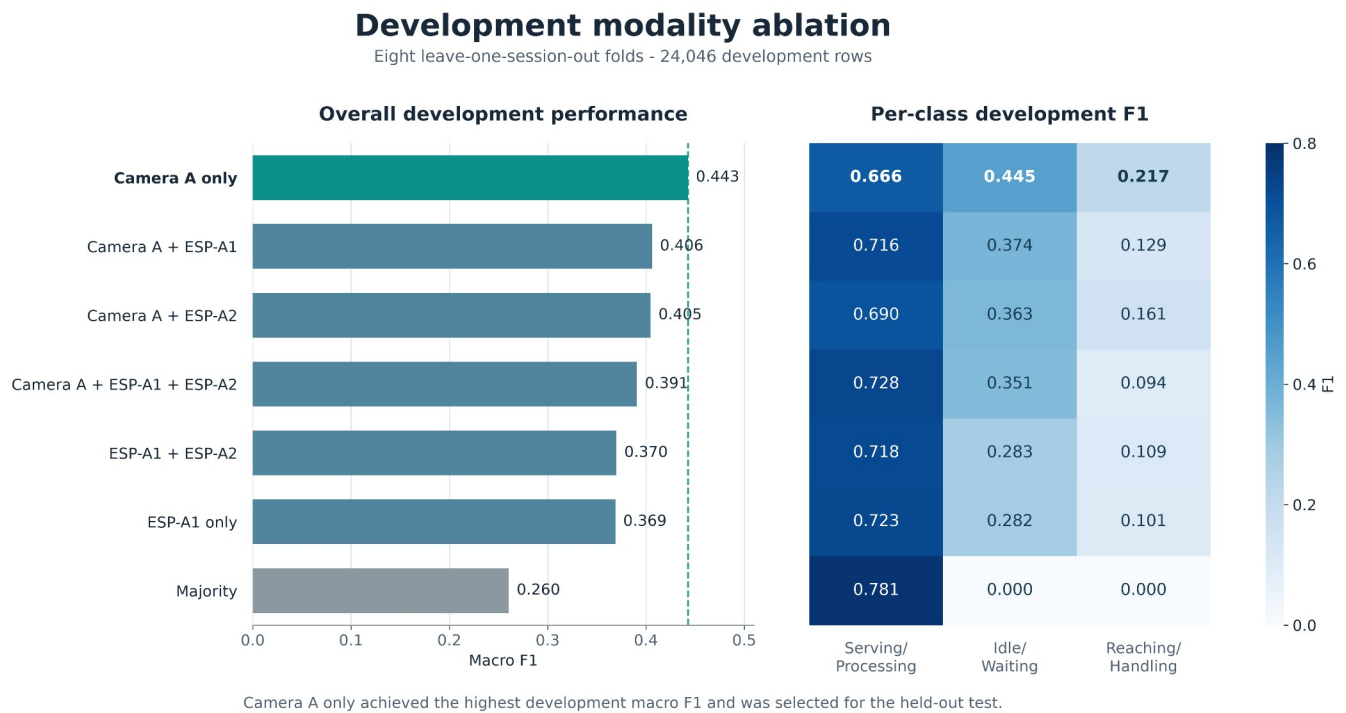


Figure 20: Frozen dev modality ablation; Camera A-only remains the selected model

For the tested Random Forest features and early-fusion configurations, Camera-only had the highest development macro F1. The strongest fused model, Camera+A1, was lower by 0.037; using both nodes increased the deficit to 0.052. Several fused models improved Serving/Processing but reduced the other classes. Shared-zone attribution noise is one plausible explanation, alongside limited sample diversity, missingness, feature averaging and the chosen model family. These mechanisms were not isolated experimentally. A separate microphone-only baseline was not retained, so this result does not establish that every audio representation or fusion architecture would fail.

The headline macro F1 pools all 24,046 out-of-fold predictions, then averages the three class F1 values; it is not the arithmetic mean of eight fold-level macro F1 scores. Figure 21 shows that Camera-only varies from 0.350 to 0.498 across held-out sessions, while Camera+A1 varies from 0.295 to 0.432. Camera-only leads in seven folds; Camera+A1 leads in one by only 0.002. These descriptive values are recalculated from the retained aggregate fold reports, without refitting models. Because the same folds were used to select

hyperparameters, they are model-selection evidence rather than an unbiased nested-validation estimate. They are not independent participant-level replicates or a confidence interval.

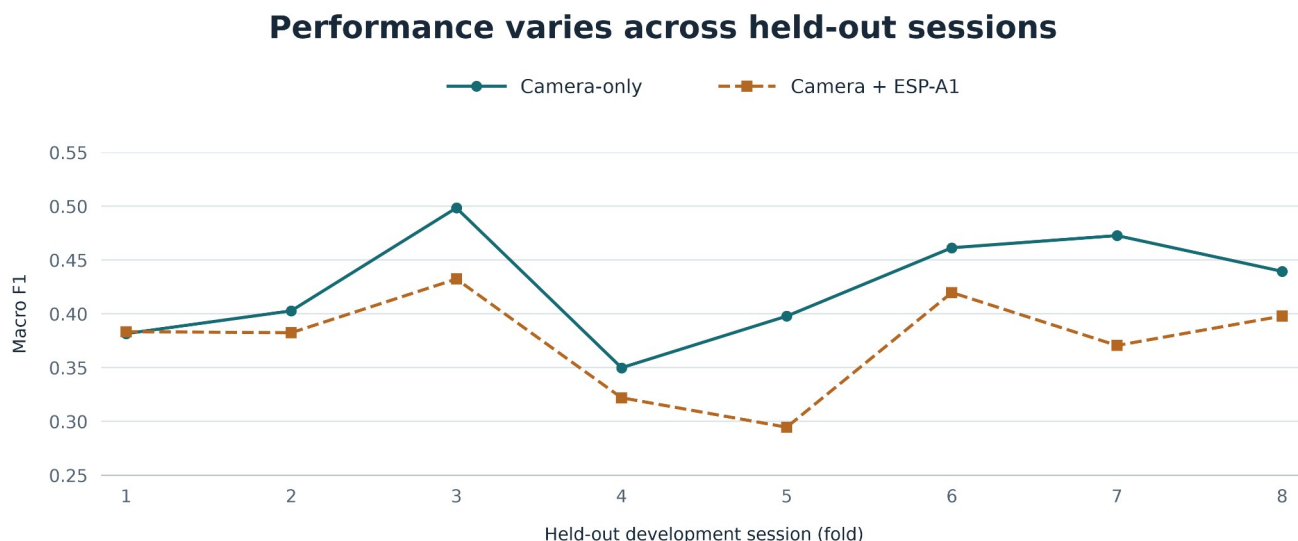


Figure 21: Retained fold-level macro F1 for Camera-only and Camera + ESP-A1

5.6. One-Time External Evaluation and Failure Analysis

Confidence is the model’s highest predicted class probability, not a verified probability that an individual prediction is correct. Expected calibration error was calculated using ten equal-width confidence bins, while the multiclass Brier score measures the mean summed squared error across the three class probabilities. Only 14 predictions had confidence of at least 0.8, and six were incorrect. This small, correlated subset does not establish a reliable high-confidence operating threshold.

Table 12: Frozen Camera-only external-test results

Metric	Result
Accuracy	0.492
Balanced accuracy	0.328
Macro precision / recall / F1	0.371 / 0.328 / 0.348
Log loss / Brier score	1.723 / 0.576
Expected calibration error	0.073
High-confidence incorrect predictions	6 of 14 predictions with confidence ≥ 0.8

Table 13: Per-class external performance

Class	Precision	Recall	F1	Support
Serving/Processing	0.645	0.564	0.602	1,093
Idle/Waiting	0.468	0.421	0.443	956
Reaching/Handling	0.000	0.000	0.000	19

External-test confusion matrix

Frozen Camera-only model - 2,068 held-out rows

		Serving/ Processing	Idle/ Waiting	Reaching/ Handling
True class	Serving/Processing	616	452	25
	Idle/Waiting	325	402	229
	Reaching/Handling	14	5	0
		Predicted class		

Figure 22: External-test confusion matrix for the frozen Camera-only model

Performance fell on the isolated external test, where macro F1 was 0.095 below the development result. Serving/Processing and Idle/Waiting remained partly distinguishable, but none of the 19 Reaching/Handling rows was classified correctly. The model also predicted Reaching/Handling for 254 rows belonging to the two common classes. The complete failure to recognise the rare class is the study's most important modelling limitation. Limited training coverage, brief movements, occlusion and similar postures are possible contributors, but the retained aggregate results do not identify their individual effects. The result was reported without external calibration or post-test tuning.

Rare-class failure includes both missed events and false alarms: 254 of 2,068 predictions (12.28%) were Reaching/Handling, although only 19 true rows (0.92%) belonged to that class, and none overlapped. The model is therefore not merely avoiding an under-represented label. Such false alarms would distort estimated activity duration even if the dashboard and tracking operated correctly. Because adjacent two-second rows may describe the same episode, these counts must not be interpreted as independent reaching events.

Table 14: Held-out performance against constant-class reference

Metric	Camera-only	Always Serving/Processing
Accuracy	49.23%	52.85%
Balanced accuracy	32.80%	33.33%
Macro F1	0.348	0.231

The constant-class reference is calculated analytically from the published class supports, not obtained by rerunning a model on the held-out data. Always predicting Serving/Processing gives 1,093 correct rows out of 2,068 (52.85%), recalls of 1, 0 and 0 (33.33% balanced accuracy), and macro F1 of 0.231. Camera-only improves macro F1 to 0.348 by recognising some Idle/Waiting activity, but its accuracy and balanced accuracy are lower. The result supports limited class discrimination rather than reliable operational recognition. This post-hoc explanatory comparison did not influence model selection or alter the one-time test result.

The selected model has a low computational cost: batch-one inference measured 2.106 ms at p50 and 2.256 ms at p95. A separate controlled-input benchmark covering feature construction, validation, inference, serialisation and local JSONL delivery measured 5.720 ms p95 after 100 warm-up iterations and across 1,000 measured iterations. These timings exclude capture, detection, tracking, pose estimation and display. They show that the measured local processing fits within a two-second window, but do not establish the delay from a physical action to its appearance on the dashboard. The live vision rate of approximately 7.5 FPS is a separate throughput constraint.

5.7. Privacy, Degraded Modes, and Delivery

Six controlled non-participant scenarios tested the deployed Camera-only inference policy. Removing ESP-A1, ESP-A2 or both left its required camera input available; removing Camera A, including loss of all modalities, produced no activity or confidence and cleared any previous prediction. This tests dependency handling, not robustness of a fused classifier to missing sensors. Fourteen dashboard scenarios and eight privacy checks covered mask blocking, stale predictions, packet-loss warnings, multiple tracks and absent media routes. Firebase checks rejected invalid writes, preserved immutable records and removed the temporary test record after verification. These software checks are distinct from the nine participant sessions used for classification.

The tests confirmed that Firebase delivery failures did not interrupt local inference. Camera unavailability or an unverified privacy mask instead caused activity output to be withheld or cleared, as intended. Privacy checks also confirmed that video, audio and identifying information were not exposed through the application's supported outputs. These findings apply to the implemented ARIA application rather than the security of the wider operating system.

5.8. Critical Evaluation Against the Research Questions

Table 15: Research-question outcomes

Research question	Outcome	Critical conclusion
RQ1: Does multimodal fusion improve recognition?	Not in the tested configurations	Tested early-fusion models underperformed Camera-only; isolated audio remains untested.
RQ2: Is local operation feasible?	Qualified yes	Inference latency is low and the 90-minute run completed, but CV throughput and network health limit deployment.
RQ3: Are camera-local identities stable?	Partial	Controlled trials include fragmentation and continuity failures; no deployment IDF1/HOTA benchmark.
RQ4: Do privacy controls fail safely?	Supported in tested scope	Mask, lifecycle, missing-modality, dashboard and cloud controls passed their documented live/synthetic checks.

The prototype is useful for investigating local activity recognition, validating privacy controls and identifying deployment failure modes. Its classification and identity errors make it unsuitable for dependable automated workload measurement, individual staff assessment or unattended operational decisions. Future work should improve rare-class coverage, test an isolated audio baseline, compare alternative temporal representations and obtain independently annotated tracking references. These are future research directions requiring appropriate approval; participant collection for this project is closed.

6. Conclusion 390/1000

6.1. Project Summary and Achievement of Objectives

ARIA produced a local Zone A prototype combining numerical ESP telemetry, masked camera processing, two-second feature windows and local activity predictions. Its procedures and software demonstration are documented; independent reproduction remains subject to restricted data access and historical-configuration limitations.

The implementation and application-level privacy objectives were achieved within the tested scope: validated inputs, mask checks, explicit unavailable states and restricted cloud summaries operated as intended. The objective of assessing multimodal recognition was achieved through comparable development ablations, but the hypothesis that fusion would improve recognition was unsupported. Camera-only achieved 0.443 development macro F1 and 0.348 on the held-out test, with no correct Reaching/Handling predictions among 19 rows.

Local operational feasibility and identity stability were only partially achieved. Model inference was fast relative to the two-second window, but vision processing remained near 7.5 FPS and network health was degraded for most of the final stability run. Controlled tracking trials also exposed fragmentation and continuity failures. The prototype therefore demonstrates a functioning research system, not dependable unattended activity measurement.

6.2. Limitations and Future Work

The single-site sample, one held-out session, correlated windows and single-annotator labels limit generalisation. Missing microphone-only and complete tracking benchmarks limit conclusions about acoustic contribution and identity stability. Shared measurements do not identify their source person, but attribution was not established as the cause of the negative fusion result.

Future work should prioritise more informative temporal features, stronger rare-class evidence, an isolated numerical-audio comparison and independently referenced tracking evaluation. Controlled network comparisons would help separate delivery problems from application faults. Any further participant study would require appropriate approval; collection for this project is closed. The current predictions should not be used for individual staff assessment or operational workload decisions.

6.3. Self-Learning and Reflection

This project taught me to distinguish successful software execution from reliable recognition. Valid records, privacy controls and visible predictions do not guarantee correct classification. The negative fusion result challenged my expectation that adding modalities would improve performance. I learned to test assumptions through comparable baselines and to value findings that contradicted my original expectations.

I also learned to make failure states explicit and use targeted tests to separate interface faults, missing inputs and modelling limitations. If I approached the project again, I would establish baselines and evaluation boundaries earlier, plan independent tracking references from the outset, and test timing, attribution and rare-class representation before increasing complexity. This would distinguish recognition improvements from software-reliability improvements.

References

- [1] Luca Arrotta, Gabriele Civitarese, Xi Chen, Julien Cumin, and Claudio Bettini. 2025. Multi-subject human activities: A survey of recognition and evaluation methods based on a formal framework. *Expert Systems with Applications* 267 (2025), Article 126178. <https://doi.org/10.1016/j.eswa.2024.126178>
- [2] Stefania Cristina, Vladimir Despotovic, Rodrigo Pérez-Rodríguez, and Slavisa Aleksic. 2024. Audio- and video-based human activity recognition systems in healthcare. *IEEE Access* 12 (2024), 8230–8245. <https://doi.org/10.1109/ACCESS.2024.3353138>
- [3] Sahar Golipoor and Stephan Sigg. 2024. RFID-based human activity recognition using multimodal convolutional neural networks. In *Proceedings of the 2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 1–6. <https://doi.org/10.1109/ETFA61755.2024.10710698>
- [4] Qimeng Li, Raffaele Gravina, Claudio Savaglio, and Giancarlo Fortino. 2023. A BSN-enabled collaborative edge-cloud architecture for multi-user activity recognition. In *Proceedings of the 2023 19th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. IEEE, 774–779. <https://doi.org/10.1109/DCOSS-IoT58021.2023.00121>
- [5] Marc Moreaux, Michael Garcia Ortiz, Isabelle Ferrané, and Frédéric Lerasle. 2019. Benchmark for Kitchen20, a daily life dataset for audio-based human action recognition. In *Proceedings of the 2019 International Conference on Content-Based Multimedia Indexing (CBMI)*. IEEE, 1–6. <https://doi.org/10.1109/CBMI.2019.8877429>
- [6] Takuya Nakabayashi and Hideo Saito. 2024. Multimodal human activity recognition on edge devices. In *Proceedings of the 2024 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*. IEEE, 136–140. <https://doi.org/10.1109/ISMAR-Adjunct64951.2024.00037>
- [7] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. 2022. ByteTrack: Multi-object tracking by associating every detection box. In *Computer Vision – ECCV 2022, Lecture Notes in Computer Science*, Vol. 13682. Springer, Cham, 1–21. https://doi.org/10.1007/978-3-031-20047-2_1
- [8] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 5–32. <https://doi.org/10.1023/A:1010933404324>
- [9] Akbar Dehghani, Tristan Glatard, and Emad Shihab. 2019. Subject Cross Validation in Human Activity Recognition. arXiv:1904.02666. <https://doi.org/10.48550/arXiv.1904.02666>
- [10] Ultralytics. Pose Estimation with Ultralytics YOLO. Official documentation. Accessed 4 September 2026. <https://docs.ultralytics.com/tasks/pose/>